

The NetBSD Security Guide

*The not so short introduction on
how to secure a NetBSD system*

The NetBSD Security Guide

*The not so short introduction on
how to secure a NetBSD system*

by Stefan Schumacher

www.net-tex.de

www.cryptomancer.de

Version 1.132, 139 pages, June 1, 2025

The NetBSD Security Guide by Stefan Schumacher
\$Header: /home/stefan/daten/dokumente/ausarbeitungen
/security-guide/security-guide.tex,v
1.132 2025/6/1 15:57:32 stefan Exp \$

Compiled at 1.132 as of 2025-6-1 15:57:32 with 139 pages.
Copyright © 2002-2005 Stefan Schumacher

“NetBSD” and “pkgsrc” are registered trademarks of the NetBSD Foundation
“Solaris” is a registered trademark of SUN Microsystems
“HP-UX” is a registered trademark of Hewlett-Packard
“BSDi” is a registered trademark of Berkeley Software Design, Inc.
“UNIX” is a registered trademark of The Open Group / X Open.
“CERT” is a registered trademark and service mark of Carnegie Mellon University.
“S/Key” is a registered trademark of Bell Communications Research, Inc.
“registered trademark” is a registered trademark of Registered Trademarking.
all copyrights and/or trademarks belong to their owner(s)

CONTENTS

o	Preface	9
o.1	About this Document	9
o.2	Where to get the Guide	10
o.3	Typesetting Conventions	10
o.4	How this Document has been Typesetted	11
i	Basic Theoretical Background	13
1	What is Security?	15
1.1	What is Security?	15
1.2	Why should a System be Attacked?	16
1.3	Common Attack Vectors	16
1.3.1	Privilege Escalation	17
1.3.2	Malicious Software ('malware')	17
1.3.2.1	Viruses	17
1.3.2.2	Worms	18
1.3.2.3	Rootkits, Backdoors and Trojans	18
1.3.3	Man-in-the-middle	18
1.3.4	Cleartext Passwords	19
1.3.5	Denial of Service	19
1.3.6	Buffer Overflow	20
1.3.7	Social Engineering	20
1.3.8	Keylogger	21
1.3.9	Compromising Radiation	21
2	Counter Tactics	23
2.1	Theory of Intrusion Detection	23
2.1.1	Data Collection	24
2.1.2	Data Analysis	24
2.1.3	Data Presentation	24
2.2	Counterintelligence with Honeypots	24
2.3	Basic Principles of Cryptography	25
2.3.1	Checksums	25
2.3.2	Symmetric Encryption	25
2.3.3	Asymmetric Encryption	25
2.3.4	Cryptographic Signatures	25
ii	Hardening Of The Base System	27
3	Hardening Of The Base System	29
3.1	First Steps	29

3.2	Zen and the Way of a Strong Password	31
3.2.1	Generating Passwords	33
3.2.2	Compute Password Hashes with OpenSSL or pwhash(1)	34
3.2.3	Checking Passwords by Cracking Them	36
3.2.4	Passwords in NetBSD	36
3.2.5	One Time Passwords with S/Key	37
3.3	Locking the Terminal	38
3.4	Daily Maintenance with daily.conf	39
3.4.1	Configuring daily.conf	40
3.5	Daily Security Checks with security.conf	41
3.5.1	En-/Disabling Tests	41
3.5.2	Modifying Tests	42
3.6	Getting Information about the System	43
3.7	System Accounting with sa(8)	45
3.8	Keeping the Clock in Sync with NTP	45
3.9	Schedule Tasks with cron, anacron or at	47
3.9.1	cron	47
3.9.2	anacron	48
3.9.3	at and batch	49
3.10	File Permissions	50
3.11	Protecting Files with Security Levels and File Flags	51
3.11.1	Kernel Security Levels	51
3.11.2	Protecting Files with File Flags	52
3.12	File System Extended Attributes (<i>NetBSD 3.0</i>)	53
3.13	Logging System Messages with syslog	54
3.14	Check for Insecure Packages with audit-packages	54
3.15	Sytem Call Policies with systrace	55
3.16	Changing the Root Directory with chroot	55
3.17	Making Backups with dump	55
3.17.1	Organising the Backup	55
3.17.2	Using dump and restore	58
3.18	Redundant Disks with RAID	59
3.18.1	Hardware RAID	59
3.18.2	Software RAID with Vinum	59
4	Fighting Malware	61
4.1	Defending Against Rootkits with Verified Executables	61
4.1.1	How to Enable it	61
4.2	Checking Filesystem Integrity with mtree(8)	63
4.2.1	What is mtree?	63
4.2.2	Example Usage	64
4.3	Checking Filesystem Integrity with AIDE	65
4.3.1	What is AIDE?	65
4.3.2	Configuring AIDE	65
4.3.3	Example Usage	68

4.4	Scanning for Viruses	69
4.4.1	Mail Virus Scanner amavis	69
4.4.2	F-prot Antivirus for Home Users	69
4.4.3	Mail Content Filtering with avcheck	69
4.4.4	ClamAV	69
4.5	Scan your Machine for Rootkits with chkrootkit	69
4.6	Scanning the Network for Zombies (dDoS Clients)	70
5	Network	73
5.1	The Internet Packet Filter ipf	73
5.2	The Internet Superserver inetd	73
5.3	Encrypted Logins and Network Connections with OpenSSH	73
5.3.1	Configuring the Server	73
5.3.2	Configuring the Client	77
5.3.3	Using OpenSSH	81
5.3.3.1	Encrypted Tunnels	81
5.3.3.2	Remote Commands	82
5.3.4	Hardening OpenSSH	82
5.4	Encrypted IP-Tunnel with stunnel	83
5.5	Encrypted Network Connections with VPN	83
5.6	Fighting Portscans with portsentry	83
5.7	Monitoring Network Hosts with arpwatch	84
5.8	Fetching Mail Encrypted with fetchmail	86
6	Cryptography	89
6.1	The Cryptographic Disk Driver cgd	89
6.1.1	cgd for a data partition	90
6.1.2	cgd for the swap partition	91
6.1.3	cgd for the tmp partition	94
6.1.4	cgd for an ISO 9660 (CDROM) Image	94
6.2	The Cryptographic Filesystem cfs	96
6.2.1	What is cfs?	96
6.2.2	Installing and Configuring cfs	96
6.2.3	Using cfs on Removable Media	97
6.2.4	Backups and cfs	98
6.2.5	cgd versus cfs	98
6.3	Symmetric Encryption with mcrypt	99
6.4	The Gnu Privacy Guard GPG	100
6.5	Deleting Data the Secure Way (“wiping”)	100
6.5.1	Disadvantages of Wiping	101
iii	Third Party Software	103
7	Third Party Software	105
7.1	The Network Intrusion Detection System snort	105
7.2	The Apache Web Server	105
7.3	The PostgreSQL Database System	105

7.4	The X Windows System	105
7.4.1	Locking the Screen	105
7.5	Web Browser and Security	106
7.5.1	Opera	107
7.5.2	Firefox	108
7.5.3	Netscape/Mozilla	108
7.5.4	Textbrowsers: lynx / links / w3m	108
7.5.5	Other Browsers	108
7.5.6	WWW Proxies	108
7.5.6.1	The Internet Junkbuster & Privoxy	109
7.5.6.2	WWWOFFLE	109
7.5.6.3	tinyproxy	109
7.5.6.4	SwiftSurf	109
7.5.6.5	SQUID	109
iv	Know Your Enemy	111
8	Attacking your System	113
8.1	Using Vulnerability Scanner	113
8.1.1	Nessus	113
8.1.2	Nikto	113
8.1.3	Hackbot	113
8.1.4	Amap	113
9	Know Your Enemy: Information about Security Holes	115
9.1	NetBSD and Security	115
10	Physical Security	117
10.1	Hardware Independent	117
10.1.1	Enclosures	117
10.1.2	Removable Disks	117
10.1.3	Redundancy	117
v	Appendices and Lists	119
A	Code & Examples	121
B	Security Relevant Sources of Information	123
C	Packages Available in pkgsrc	125
D	Bibliography	129
	Index	137



PREFACE

“ERFAHRUNG IST VERSTANDENE WAHRNEHMUNG.”
Immanuel Kant

0.1 ABOUT THIS DOCUMENT

This document is not an official publication of The NetBSD Foundation or NetBSD project. It has been written in my leisure time just for the fun of writing it.

This paper is an attempt to publish information about security problems with NetBSD (and sometimes Operating Systems in general) and how these problems can be solved. Additionally, I want to introduce third party software that has been developed to improve security or has to be secured, respectively.

This Guide begun as an essay to bundle my articles regarding NetBSD and security. Once it grew to large to be a simple essay, I wrote more chapters to get some kind of guide.

It handles NetBSD 2.0.2, if -current is used, it is marked.

Non-commercial and personal use and redistribution of the guide is permitted, provided that it remains unmodified, that means you are not allowed to modify the content and/or add or remove pages.

However, printing for personal use or for NetBSD advocacy use is permitted. A fee to cover printing coasts may be demanded.

Mass production, commercial use, or the use of a modified copy requires explicit written permission. Permission may be obtained by writing to:

If you find this document useful (or not), drop me a line via email or postcard.

0.2 WHERE TO GET THE GUIDE

The Security Guide has been published first on my personal homepage <http://www.net-tex.de/sg/> and new versions will show up there first.

There will be the change log on my site, so one can see the version history.

Additionally, the PDF file available at my website will be signed with my OpenPGP key, which is 1024D/B3FBAE33 or CF74 D5F2 4871 3E5C FFFE 0130 11F4 C41E B3FB AE33

0.3 TYPESETTING CONVENTIONS

Commandlines are set in a shadowbox and in monospaced fonts. The prompt determines if it is a usual user's prompt (\$) or a root prompt (#). If a command is too long, it can be broken with a single backslash ('\') to a new line.

```
$ this is a user prompt
# this is root's prompt
```

Example 1: Showing an Example

On larger scripts, the lines are numbered to make citations easier.

URLs, references and citations are linked to the target, so you can click on them.

Package or filenames are set in monospace fonts. Sometimes, I use the `command(section)` (eg `ls(1)`) notation, the number in brackets determines the section of the regarding man page, for further information, see [man . man](#) Man pages are linked to the [gw.com](#) man page web interface at <http://netbsd.gw.com/cgi-bin/man-cgi>.

[ppo]

A marginal note with a doublebox and [pp#] refers to a third party package that is available in pkgsrc. The [pp#] is linked to the appended list. At page 125 is a list of all referenced packages. A package entry lists the package's number, name, the short description, its path in pkgsrc, the pkgsrc website and finally the project's homepage.

Bibliography entries marked with a (*) are in German, others are in English.

0.4 HOW THIS DOCUMENT HAS BEEN TYPESETTED

This document has been created using pdf $\LaTeX$ and optimized into a “Linearized PDF” by pdf $\LaTeX$ (1).

I used `scrreprt` of the KOMA-Script package by Markus KOHM et. al. with the options:

`titlepage`, `makeidx`, `a4paper`, `idxtotoc`, `liststotoc`, `twoside`, `openright` and the pdfspecials offered by pdf $\LaTeX$.

Additionally I used `scrpage2`, `longtable`, `fancybox`, `enumerate`, `float`, `alltt`, `scrdate`, `textcomp` and `makeidx`.

The sourcecode has been written with `gvim` using a lot of `.vimrc`-makros that can be found in my `.vimrc` at my homepage.

I use `CVS` to manage revisions of the document and `NetBSD-make` to compile the source to PDF.

Part 1

BASIC THEORETICAL BACKGROUND

WHAT IS SECURITY?

WIR MÜSSEN WISSEN - WIR WERDEN WISSEN.
David Hilbert

1.1 WHAT IS SECURITY?

[DIN61508](#) describes security as a condition, which is free from untenable risks of damage or is free of hazards.

An operating system has to be, at any given time, in a well defined condition. This condition has to be determined by the system manager or user and must not be influenced by illegal access.

Computer security covers several aims:

- data privacy
 - data confidentiality
 - privacy
 - transmission reliability
- data security
 - integrity of services
 - functionality
 - non-repudiability

- authenticity
- also covered
 - access control
 - availability of services
 - infeasibility of services (sth. has been done)

1.2 WHY SHOULD A SYSTEM BE ATTACKED?

The reasons for attacking a system can vary in a wide range, from a simple and stupid act of vandalism performed by Script-Kiddies to a well organized crime committed by professional intruders. It is very important to understand how systems can be penetrated if you want to defend them - there is no better defender then a well trained hacker.

The most common way to penetrate systems are nowadays insecure network services, either the running program is broken or it has been wrong configured. NetBSD offers several builtin features to enhance the security of a running system and additionally there are several free packages available that are useful too.

This paper is intended as an introduction to the field of OS-security. Security is a very complex evolving process and not a status that can be achieved by installing a program. If you are really interested in securing your machines, you have to gain deeper knowledge of how NetBSD, networks and networkservices work and how something can be exploited by an intruder. You are also required to spend time on further education and updating your systems.

1.3 COMMON ATTACK VECTORS

There are several ways of attacking a system. Attacking a system means to gain superuser access to the system's resources, so the intruder can control it.

1.3.1 *Privilege Escalation*

This means a normal user gains root-access by exploiting a program that has root-privileges. One example is `/usr/bin/passwd`, the program that allows any user to change his password. It must be executable by any user, but it must also have root privileges to update the `/etc/passwd` file. If a user is able to hijack the program to generate a privilege escalation, he will get root-privileges and is therefore able to manipulate the whole system.

1.3.2 *Malicious Software ('malware')*

In general, malware is any kind of software built to harm other software, like operating systems. There is no strict classification of malware, but I'll give a short overview of several types existing.

Malware usually is some kind of executable program, so it can be fought with several techniques, described in this paper.

1.3.2.1 *Viruses*

A Virus is a program that is able to reproduce itself and infect other computers. A Virus requires a host for transmission, like an executable file, a boot sector or a script. Viruses are currently focussed on a commercial family of operating systems, but NetBSD could be attacked, too.

In general, a virus can be found by a virus scanner. Virus scanner use pattern matching techniques and / or heuristics to detect a virus within a host, so the virus has to be analyzed first to be recognized.

Since a virus is a program, it can be fought by preventing its execution with verified executables.

A virus might be destructive, but there are also a lot of viruses being simply annoying.

Nowadays, viruses very often spread via emails, usually exploiting a substandard email client designed to enable virus replication. It might be a good idea to scan emails delivered through a NetBSD server.

1.3.2.2 *Worms*

A worm is similar to a virus, but it's replicating itself over network, not a host. Usually, worms infect network services and exploit vulnerabilities in them to gain access to a system. Once the system has been infected, the search for other victims.

Worms usually carry destructive programs in their payload, but even if not, they harm the network by the pure traffic generated.

Worms can be fought with the same techniques as viruses and additionally, network services should be protected eg. with a packet filter

1.3.2.3 *Rootkits, Backdoors and Trojans*

A rootkit is a program usually installed after a successful penetration of a system. It is a set of manipulated binaries (eg `ls`, `who` or `ps`) that are patched to keep the intruders resources and presence hidden from root. Usually a rootkit also installs a backdoor to allow easy access to the system, eg a manipulated `sshd`. Sometimes a rootkit (or a trojan) also tries to spy on users by recording keyboardinput or making screenshots.

Rootkits, backdoors and trojans often use worm and/or virus methods to spread, so they can be fought with the same methods described above.

SEE ALSO:

Chapter 4.4, "[Scanning for Viruses](#)" on page 69

Chapter 4.5, "[Scan your Machine for Rootkits with `chkrootkit`](#)" on page 69

Chapter 4.2, "[Checking Filesystem Integrity with `mtree\(8\)`](#)" on page 63

Chapter 4.3, "[Checking Filesystem Integrity with `AIDE`](#)" on page 65

Chapter 4.1, "[Defending Against Rootkits with `Verified Executables`](#)" on page 61

Chapter 5.1, "[The Internet Packet Filter `ipf`](#)" on page 73

1.3.3 *Man-in-the-middle*

A 'Man-in-the-middle'-attack (Mitm) is an attack where an intruder is able to read and manipulate transmissions between two points of communication. Mitm applies to communication over networks, esp. when no security mechanisms like cryptography is used.

An eMail can be intercepted, manipulated and be forward to its original recipient but recovering the manipulation. This problem affects any form of communication and requires strong authentication and encrypting technologies.

1.3.4 *Cleartext Passwords*

Some services (ftp, telnet, rsh, http) transmit required passwords in cleartext over a network. This is no problem if only both communication partners are on the line, but a switching network can be compromised and sniffed.

All services using cleartext passwords should be abandoned or substituted by more secure services (eg. OpenSSH, OpenSSH or https). Additionally those services can be wrapped by a cryptographic tunnel.

SEE ALSO:

Chapter 5.3, "[Encrypted Logins and Network Connections with OpenSSH](#)" on page 73

Chapter 5.4, "[Encrypted IP-Tunnel with stunnel](#)" on page 83

Chapter 5.5, "[Encrypted Network Connections with VPN](#)" on page 83

Chapter 7.2, "[The Apache Web Server](#)" on page 105

1.3.5 *Denial of Service*

A denial of service is an attack where a service of the victim is caused to be unavailable. This can be caused by exhausting the available network traffic of the victim, usually using so called zombies like 'Stacheldraht'. Stacheldraht exploited buffer overflows on several Unix OS (Solaris 2.x, HP/UX, BSDi, ...) to spread on the internet. Once the zombies were installed, they contacted a master and awaited order from him. The zombies could be ordered to 'syn flood' a target to exhaust the available TCP/IP connections of the victim. At that time (1999) no one really expected that kind of distributed denial of service (DDoS) and major ecommerce sites were attacked causing them to be unavailable.

Nowadays, a lot of DDoS attacks have been analyzed and most of them can be fought (eg. with SYN cache).

Fighting zombies requires security mechanisms on other OS too, so they will exist for a long time.

1.3.6 Buffer Overflow

1.3.7 Social Engineering

Social engineering is an intrusion technique¹, where the intruder tries to trick it's victim into revealing useful information. With that knowlege, he tries to trick another person or he tarts attacking the network itself.

It's a very effective method esp. if the intruder wants to attack a large organization. Usually, the intruder calls an employee and feigns beeing eg. an co-worker, administrator or technician to start a talk about security measures. In this talk, the intruder tries to gather relevant information eg. used software, security policies and so on.

A currently wide spread problem of social engineering is "phishing²", where fake-mails are sent pretending to be from a bank or internet shop. The recipients are tricked into believing the mail is official and reveal their password.

There is no technical solution³ against social engineering, since it is aimed at *social* problems. The only way to prevent those attacks is to instruct users and establish security policies. Additionally, one should keep in mind that many attacks also origin *inside* an organisation.

Some articles of a security policy should be:

- restrict access to vulnerable parts of the network
- restrict physical access to the network⁴
- restrict user rights as much as possible (without constraining them, of course)
- instruct users about risks of social engineering
- secure the hardware

SEE ALSO:

¹ Originally, the name was introduced in political science to describe the engineering of citizens behaviour.

² password harvesting fishing

³ 'If you think technology can solve your security problems, then you don't understand the problems and you don't understand the technology' - Bruce Schneier

⁴ Heckler & Koch produce valuable security enhancements

km1
km2
wg
sef
se1

1.3.8 *Keylogger*

Nowadays, so called keyloggers (programs that log *any* keystroke) are available either as software or as hardware. Software keylogger can be fought with the same procedures as viruses or rootkits. But there are also hardware devices available to be connected to a keyboard. Those devices record any keystroke and can of course be used to gather all passwords or other vital information. Hardware keylogger are software independent and are usually *very* small.

Examine your keyboard plug to detect them. As far as I know, only USB and PS2 or DIN Plugs are available, no RJ6 or SUN connectors have been seen in the wild.

Thinkgeek.com has a keylogger (with photos) available at
<http://www.thinkgeek.com/gadgets/security/5a05/>

1.3.9 *Compromising Radiation*

Electromagnetical radiation is emitted by any electrical device - be it a TV, a microwave oven, a TFT or CRT, a network cable or a simple keyboard. This radiation can be received, decoded and recorded even across large distances. Additionally, it is possible to recognize the transmitted and / or processed data.

The required equipment is indeed somewhat professional and expensive, so it's not in use within the script kiddie scene.

Fighting compromising radiation is also a complex and expensive field of security. There are some devices available that work as a jammer to disturb radiation eg. of monitors, and make it useless for sniffer. Also, there are devices available with special shielding technology to lower radiation.

Radio technology laboratories often use special metal alloy plates to shield radiation, this shields can be built in into the facility. Furterhmore, special wallpapers that are combined with metal grid to span a Faraday cage are available an can be use to shield a room.

Network cables made of metal (twisted pair, fieldbus, printer cable and so on) radiate, too. If those cables are exposed to positions where they can be sniffed but not shielded, cryptographic tunnels should be established. Optical fibre cables are harder to sniff, especially since there is only one method, developed by Deutsche Telekom AG, to sniff data without damping the signal strength by using Rayleigh scattering.

Wireless keyboards and mice should of course be avoided, too.

Wireless LAN radiation could be limited by using beam antennas instead of isotropic sources.

SEE ALSO:

[rad](#)

“DAS WISSEN MUSS EIN KÖNNEN WERDEN.”
Carl von Clausewitz

2.1 THEORY OF INTRUSION DETECTION

Intrusion Detection Systems (IDS) are systems designed to detect intrusion attempts or successful intrusions. Additionally, some IDS can be used as intrusion prevention system or forensic tool.

An IDS has to react on several events, like malware, privilege escalation or attacks against network services. It is a useful tool to raise network security, but it is definitely not a magic bullet. It can only be embedded into other security mechanisms, such as social policies, access restriction or cryptography. Additionally, an IDS is rather complex and requires a lot of knowledge of the operating system and network protocols/services used. It is not some kind of snake oil, that can be spread within the net, but a complex system requiring a mature strategy.

An IDS consists of three components:

- data collection
The IDS has to collect generated data and prepare it for analysis.
- data analysis
Collected data has to be examined regarding possible or occurred attacks.
- data presentation
The output has to be presented.

Additionally, one can divide IDS into several classes according to their:

reaction time:

- proactive (before an intrusion can succeed)
- reactive (after an intrusion has succeeded)
- post mortem (forensical analysis)

or placement:

- host based
- network based
- hybrid

2.1.1 *Data Collection*

Data collection is not only a technical problem, but also a social one. Although it is not a problem to collect any data available

2.1.2 *Data Analysis*

2.1.3 *Data Presentation*

SEE ALSO
[ids](#)

2.2 COUNTERINTELLIGENCE WITH HONEYPOTS

SEE ALSO
[ids](#)

2.3 BASIC PRINCIPLES OF CRYPTOGRAPHY

2.3.1 *Checksums*

2.3.2 *Symmetric Encryption*

2.3.3 *Asymmetric Encryption*

2.3.4 *Cryptographic Signatures*

SEE ALSO

[krypt](#)

Part II

HARDENING OF THE BASE SYSTEM

HARDENING OF THE BASE SYSTEM

“DER COMPUTER RECHNET MIT ALLEM,
NUR NICHT MIT SEINEM BESITZER.”
Dieter Hildebrandt

3.1 FIRST STEPS

The first steps towards a more secure system regard the base system. There are several rules to follow, which I will describe in this chapter.

LEAN SYSTEM

Any engineer knows: the more complex a system is, the harder it is to manage. This applies to operating systems too. It is a good strategy to make an OS as lean as possible, this means one should not run services, that are not required.

In general, NetBSD follows this philosophy of course. A base system only installs a minimum set of required services and does not enable them by default.

Do not install sets you do not need (eg. “X” or “games” on a server). These knowledge extends to packages as well.

Don’t run services not required. `/etc/rc.conf` is used to start (or not start) services. [man 8 rc.conf](#) explains which options are available and how services can be started. Additionally, there is `/etc/defaults/rc.conf` as an commented example of a valid `/etc/rc.conf`.

A lean system also requires a lean kernel. It is a good idea to disable kernel drivers and modules not required. You have no SCSI and NFS? Remove them. `adjustkernel` [\[pp20\]](#) can be used to generate a kernel configuration that is adjusted to your hardware. The options of kernel configuration are described in [man 8 . options](#)

[pp20]

The NetBSD kernel supports so called loadable kernel modules. This technology allows root to add modules (eg. a driver or support for a new filesystem and so on) to the kernel. A module loaded into the kernel can manipulate the whole kernel, so it can compromise the whole system. Once an intruder gains control of `/dev/lkm`, he can control the whole system by manipulating the kernel via modules.

LKM can be disabled in the kernel configuration, by commenting out
`options LKM# loadable kernel modules`
 or rising the kernel security level to a level greater than zero.

UP-TO-DATE SYSTEM

A system should be up to date, especially if security holes have been confirmed. The NetBSD Project established several mechanisms to handle security flaws in the base and in pkgsrc-Packages. If flaws in the base are detected and fixed, an email will be sent to the `netbsd-announce` mailing list. For pkgsrc packages, a program called `audit-packages` [pp17] exists. It downloads a machine parsable list of security advisories and compares it to the running system. Exploited packages are listed and should be replaced at once.

[pp16]

[pp17]

Also there is `pkg_chk` [pp16] available. It compares installed packages against available versions in pkgsrc. So all packages that can be updated are listed.

```
$ pkg_chk -i
glib2-2.6.6nb1: version mismatch - glib2-2.6.4
```

Example 1: check versions of installed packages

NEVER WORK AS ROOT

Never ever log in and work as root. There is nothing root can not do, so it's easy to blow away your whole leg.

[pp18]

Use `su(1)` or the package `sudo` [pp18], which allows other users to run commands as root.

SEE ALSO:

`man rc.conf` `man rc.d` `man su` `man options` `man lkm`
 [Part II. System installation and related issues]guide
 [Part V. Building the system]guide
 Chapter [sec](#), "Bibliography" on page 132
 Chapter 3.6, "Getting Information about the System" on page 43
<http://netbsd.org/MailingLists/#netbsd-announce>
[lkm phrack-lkm](#)

3.2 ZEN AND THE WAY OF A STRONG PASSWORD

The key to a NetBSD computer is a password. With that password, a user is identified at login and will be granted all privileges the user has. If an untrustworthy person learns your password, he will get access to the machine and will misuse it's resources.

What is a Strong Password?

To protect your computer, you will need a strong password for any account available. So first we have to clear out what a strong password is, by thinking like an intruder.

The first and easiest attempt to break into an account is to guess the password. People tend to use things they can remember, like the name of their pet, hometown, mother, children and so on. So it's usually worth a try to simply guess the password. If the intruder has more time and will to break into your account, he also might use social engineering techniques to gather more information¹ about you.

To avoid trial-and-error guessing of passwords, it's nowadays a standard to lock the login routine if several logins failed. NetBSD will pause for 15 seconds after 5 failed login attempts.

The first rule of a strong password is:

Don't use a word that can be guessed!

The password for an account is stored in `/etc/master.passwd` and compared to what an user enters at login. The password string itself is stored in encrypted form, to avoid it's revelation if an intruder gets access to the password database. If an intruder gets access to the encrypted password string², he won't be able to compute the cleartext, as long as the used cryptographic algorithm is secure. But it is possible to recover the cleartext with a 'dictionary attack'.

A dictionary attack is performed in a very simple way:.. feed a dictionary word by word to the encryption program and compare the result - if the generated string equals the password string, the password is found. So the second rule of a strong password is:

Don't use a word that can be found in a dictionary!

The above described dictionary attack can also be driven by a generator instead a wordlist. This generator simply tries to generate words that could be used as a

¹ You have a weblog? You publish your dog's name there? He will try it as a password.

² eg.: "3e9e1f6f189ff8e4d505c81b54a796d5e66a2542" is "NetBSD" encrypted with MD5

password, like 'John1980' or 'Rover#1', so it is important to use a long password with capitalization, numbers and special characters.

If an intruder wants to generate all possible passwords, he has to compute

$$\text{available letters}^{\text{password length}}$$

possible combinations.

If your password contains of a-z (26 letters) and 6 digits, he has to compute $26^6 = 308'915'776$ combinations, which can be achieved on modern computers within hours. But if you choose a-z, A-Z and 0-9 (62 letters) and 10 digits, he has to compute $839'299'365'868'340'224$ combinations. Benchmarks say, a Pentium I/100 MHz will need 34 minutes to compute 26^5 combinations, which is indeed very short. If you add a single digit, the computer will need 15 hours, but if you use 8 digits and 62 letters, the machine will compute about 1000 years! So we can say the next important rules are:

Use a long password!
Use capitalization, numbers and special characters!

And last but not least, you have to keep your password private. Don't write it down where it can be uncovered (eg, under the mousepad or keyboard). If you have to write it down, keep it somewhere where an intruder cannot access it, eg your personal safe. And don't tell it anyone! If you are a user on a multiuser NetBSD-Server not administrated by you, even root won't need your password! So the last rule of strong passwords is the simplest:

Keep the password a secret!

To summarise the rules of a strong password:

- Don't use a word that can be guessed! (*pet/wife/children name*)
- Don't use a word that can be found in a dictionary!
- Use a long password! (10 letters)
- Use capitalization, numbers and special characters (+, *, β, -, €)
- Keep it a secret!

3.2.1 *Generating Passwords*

To generate a password isn't simple, if you want a strong one. But with some help, either from an algorithm or a program, it's easy to find an appropriate one.

The first way to create a password is to use a sentence, that is shortened to the initials, somewhat like an akronym:

- crate a long sentence, that you will remember easily
- shorten it to make it a password
- substitute some letters

An Example:

*Whoever has succeeded in the great attempt,
To be a friend of a friend - Friedrich Schiller* would become:

Whsitga, Tbaf0af – FS

additionally, you could substitute some letters with special characters or numbers³ to make it more complex, eg. a=4, o=0, i=1 leads to:

Whs1tg4, Tb4f04f – FS

what is indeed a really strong password. All you have to remember is the sentence and the substitution rules.

Another way is to mix up a word with numbers and special characters like that:

RingParable → R1i2n3g4P5a6r7a8b9l0e

instead counting from 0 up to 9, you could use eg. a name and the person's birthday:

2L2e0s1s1i7n2g9

Automated Password Generator apg

apg is a program to generate passwords. It can use two different algorithms, one to

[pp3]

³ That's 31337, isn't it?

generate pronounceable passwords, as described in [FIPS-181](#) , and another one to generate random character passwords.

To generate pronounceable passwords with a length between 8 and 12 characters use:

```
$ apg -a 0 -m 8 -x 12
DahyRijHypjo
wrisEij8
Nuphyeulvuk
itsAcugHyd9
kribazGuc
OtOknelun
$
```

Example 2: password generation with apg

apg offers several options to influence the way passwords are generated. Using a long and really random generated password will enhance security - but it will be hard to memorize. So it's a good idea to create either a pronounceable password or one on your own.

However, apg is a useful tool if one has absolute no idea of a new password - especially if you have to create new accounts for multiple users.

If apg seems to hang while it generates passwords, it needs some activity for the random number generator. This can be made with a simple `find /.`

3.2.2 Compute Password Hashes with OpenSSL or *pwhash(1)*

OpenSSL offers the `passwd` command to compute several forms of password hashes. It can read a password either from stdin or from a file, what is useful in shellscripts. Currently, "crypt", "BSD MD5" and "Apache MD5" are supported, that means an account whose password hash shall be generated must use either 'localcipher=old' or 'localcipher = md5' in `/etc/passwd.conf`.

```
$ openssl passwd -salt AA MyPassword
Warning: truncating password to 8 characters
AAaV85M5Ph8j6

$ openssl passwd -table -1 -in userlist
foo      $1$C9/uTqZa$s1/X4ihpfvG/.m2EYzbZV1
bar      $1$7NDpX3zi$mBKk39Z9k6s4zbBFkPYt4.
baz      $1$bZywpxFq$xyjJAsf6MW.pX3VohvwJE.
$
```

Example 3: generate passwords with OpenSSL

The first command generates an old crypt-style hash, using ‘AA’ as salt and the first eight letters of ‘MyPassword’ as password. The second command produces a table of password and hash reading the passwords from file ‘userlist’ generating BSD style MD5 hashes.

pwhash(1) is a NetBSD program to generate password hashes from command line (which is discouraged) or standard input. It can generate all sorts of hashes available in NetBSD, as described below:

```
$ pwhash -b 20
test
$2a$04$gkUo0P4qyxvF0iUdXIqQ9uAwZa7o.oeg.4d8uax67mD6/...
$ pwhash -m md5test
$1$a4w7f0jp$/U7Flz7YAx77KaFl2Grnl.
$
# useradd -g mygroup -d /home/testuser -m -p \
    'pwhash -b 9 ?MyP455W0Rd!' testuser
#
```

Example 4: password hashes with pwhash

The first line generates a Blowfish hash using 20 rounds, the second generates MD5. The last command can be issued by root to add a new user with the specified password.

pwhash can be used to make a little benchmark of password ciphers, I generated a password with n blowfish rounds using ksh-time to take the time. The computer used is a i386 AMD Athlon K7 1000 MHz with 1500 MB Ram:

```
# for i in 4 5 10 15 20; do
; time pwhash -b $i pwhash; done

04 Rounds:    0.01s real    0.01s user  0.00s system
05 Rounds:    0.02s real    0.01s user  0.00s system
10 Rounds:    0.30s real    0.27s user  0.00s system
15 Rounds:    9.12s real    8.42s user  0.00s system
20 Rounds:   305.80s real  269.94s user  0.22s system
```

Example 5: benchmarking pwhash

A higher time is good to fight dictionary attacks, but you also have to wait this time at each login. So it should be sufficient to use something between 10 and 15 rounds.

3.2.3 *Checking Passwords by Cracking Them*

[pp1]

crack

[pp4]

hydra

[pp5]

john

3.2.4 *Passwords in NetBSD*

Password hashes are stored in `/etc/passwd` and `/etc/master.passwd`. `/etc/master.passwd` contains of colon seperated fields containing login, password, User ID, Group ID, login class, password change time, expiration time, general information, home directory and finally the login shell. `/etc/passwd` is generated from `/etc/master.passwd`, removing login class, password change time and expiration time and substituting the password by a single asterisk (*).

The crypto algorithm used to generate the password hash can be determined using `/etc/passwd.conf`.

It contains the configuration of the password ciphers in simple plain text format:

```
default:
    localcipher = blowfish,7
    ypcipher = md5

myuser:
    localcipher = blowfish,9
    ypcipher = blowfish,9

:mygroup:
    localcipher = blowfish,7
    ypcipher = newsalt,9

root:
    localcipher = blowfish,25
    ypcipher=blowfish,25
```

Example 6: /etc/passwd.conf

The first option is the user's login (or group, if a single colon [':'] is prefixed), followed by a colon. localcipher determines the local password (/etc/master.passwd) while ypcipher determines YP (NIS) passwords. Finally, the desired cipher (and the rounds) are supplied. ypcipher and localcipher can differ, but don't have to.

Currently, the available ciphers are:

- old
- newsalt,n, where n is the number of rounds
- md5
- blowfish,n, where n is the number of 2^n rounds

old, newsalt and md5 should be avoided, blowfish (a cipher designed by Bruce Schneier) can be considered the best cipher.

3.2.5 One Time Passwords with S/Key

S/Key implements one time passwords based on a challenge / response mechanism. First, n passwords have to be generated during a setup process. The setup process must be initiated in a secure environment, eg. a direct login from the console or via SSH from a trusted client.

First, a secret key has to be provided, either by the user or from a random number/word generator like `apg`. This secret key is the heart of the authentication and should not be compromised at all. If it is discovered, the whole S/Key process is compromised.

A cryptographic hash (MD4, MD5 or SHA1) is used to generate `n` hashes over the provided secret key. These `n` hashes are the one time passwords, listed in reverse order. The first password (`hash(secretkey)`) is stored on the server, all other hashes and the secret key is dropped.

You are required to write the passwords down on a sheet of paper.

To login to a server, you have to provide the password

SEE ALSO:

[skey-de](#)
[rfc1760](#)

3.3 LOCKING THE TERMINAL

Sometimes it is required to lock the terminal without logging out. There is `lock(1)` to lock a terminal. `lock` can take a timeout (`-t minutes`) or no timeout (`-n`). By default, `lock` requires the user to issue a key at logout, this can be changed to use the NetBSD (or S/Key) password instead with `-p`.

This will lock the terminal without timeout using the NetBSD password:

```
$ lock -n -p
lock: /dev/ttyp5 on wieland.net-tex.de. no timeout.
time now is Thu Sep 29 21:23:33 CEST 2005
Key:
```

Example 7: locking the terminal with `lock`

To let a program run while your terminal is locked, you can add an ampersand (`&`) to the command. So the process will run in the background, sending it's messages to `stdout` and `stderr`. To avoid getting messages use `1&>2` to redirect `stderr` and `stdout` to `/dev/null`.

If a program shall run even when you really log out, use `nohup(1)`, which will

intercept HUPs to the process, keeping it running when logging off. `nohup` will automatically redirect the program's output to `./nohup.out`. Add a `&` to send the program to the background.

```
$ cvs update -dP 1&>2&
[1] 18998
$ clear
[1]+  Exit 1                  cvs update -dP 1 >&2
$ nohup cvs update -dP &
```

Example 8: running a process in the background with `&` and `nohup`

The first command starts a CVS update in the background, redirecting the messages to `/dev/null`. After the process finished and another program is run, the shell will give a short status message. If you logout while `cvs` is running, it will stop. The second line uses `nohup` and a `&` to let `cvs` run in the background, even if you log out. The messages of `cvs` are sent to `./nohup`.

Another very cool tool for a lot of Unix OS is `screen`. It is available via `pkgsrc` and works as a terminal multiplexer. So if you run `screen`, you can use multiple shells. This is very useful when logging in via SSH to a server without X. But another very useful feature of `screen` is the ability to detach it. A detached session will run in the background and can be reattached. This is useful if you want to keep your work status visible even after logging of or if you want to keep a program running. Any program currently running in `screen` will run further, it's messages are sent to the `screen` terminal, so they can be examined when reattaching the `screen`. `screen` offers a lot of other useful features, too, so check out the man page.

SEE ALSO:

[man](#)
[lock man](#)
[nohup man](#)
[signals man](#)
[screen](#)

3.4 DAILY MAINTENANCE WITH `DAILY.CONF`

By default, NetBSD runs maintenance scripts defined in `/etc/daily` every night.

The script is by default started 03:15 via `cron`.

If your machine does not run around the clock, you can use anacron (see Chapter 3.9.2) instead of cron.

```
# do daily/weekly/monthly maintenance
15      3      *      *      *
    /bin/sh /etc/daily 2>&1 | tee /var/log/daily.out | sendmail -t

30      4      *      *      6
    /bin/sh /etc/weekly 2>&1 | tee /var/log/weekly.out | sendmail -t

30      5      1      *      *
    /bin/sh /etc/monthly 2>&1 | tee /var/log/monthly.out | sendmail -t
```

Example 9: crontab-entries for maintenance scripts

Additionally to `/etc/daily` two other scripts exist. `/etc/weekly` is run 04:30 each Saturday, and `/etc/monthly` which is run 05:30 the first day of a month.

As of NetBSD 2.0.2, `/etc/monthly` starts no tasks, while `/etc/weekly` cleans the UUCP spool and rebuilds the locate database.

3.4.1 *Configuring daily.conf*

`/etc/daily` is configured via `/etc/daily.conf`, where variables can be set to “YES” or “NO”. An example can be found at `/etc/defaults/daily.conf`, but this file should not be edited, since it can be updated at system updates.

The variables and tests of `/etc/daily` are:

`CHECK_DISKS` reports disk and raid status with `df(1)` and `dump(8)`

`CHECK_MAILQ` uses `mailq(1)` to print the mail queue

`CHECK_NETWORK` runs `netstat -i`, `ruptime` and checks the `rwhod` database

`CHECK_UUCP` run `/etc/uuclean.daily` and `uustat -a`

`EXPIRE_NEWS` run `/etc/expire.news`

`FIND_CORE` `find(1)` is used to find coredumps (`*.core`) on /

`FULL_NETSTAT` uses `netstat -inv` in `check_network` to generate verbose netstatus

`PURGE_ACCOUNTING` purges accounting logs in `/var/account`

`RUN_CALENDAR` invokes `calendar -a`, all calendar files are processed and mailed to the user

`RUN_FSCK` runs `fsck -n` (assumes “no” as answer)

`RUN_MSGS` cleans `/var/messages` with `msgs(1)`

`RUN_RDIST` runs `rdist -f /etc/Distfile` to keep specified directories on listed hosts in sync

`RUN_SECURITY` runs `/etc/security` (see 3.5)

`RUN_SKEYAUDIT` runs `skeyaudit` to check S/Keys (one time passwords) for expiring

`SEND_EMPTY_SECURITY` sends even empty security reports to root

`SHOW_REMOTE_FS` includes remote filesystem in `check_disks`

SEE ALSO:

[man](#)

[daily.conf](#) [man](#)

[cron](#) Chapter 3.9.2, “[anacron](#)” on page 48

[man](#)

[anacron](#)

3.5 DAILY SECURITY CHECKS WITH `SECURITY.CONF`

`security.conf` is NetBSD’s way to check several security relevant issues each night. By default, the tests are configured via `/etc/security.conf` and started as the `run_security` service of `daily.conf`

3.5.1 *En-/Disabling Tests*

`CHECK_PASSWD` check `/etc/master.passwd` for inconsistencies

`CHECK_GROUP` checks `/etc/group` for inconsistencies.

`CHECK_ROOTDOTFILES` check root's dotfiles for sane \$PATH and umask.

`CHECK_FTPUSERS` checks /etc/ftpusers for correct user settings

`CHECK_ALIASES` checks /etc/mail/aliases and /etc/aliases for security problems

`CHECK_RHOSTS` check .rhosts and hosts.equiv for "+" (a joker)

`CHECK_HOMES` check homes for correct owner/persmissions

`CHECK_VARMAIL` check /var/mail for correct owner/persmissions

`CHECK_NFS` check if /etc/exports exports filesystems to the world

`CHECK_DEVICES` check devices and setuid files for changes

`CHECK_MTREE` run mtree to check the filesystem integrity, mtree is controlled by the files in /etc/mtree/

`CHECK_DISKLABELS` backup disklabels and fdisk information of disk to /var/backups/work/ and check them for changes

`CHECK_PKGS` generates a list of packages to /var/backups/work/pkgs and checks it for changes

`CHECK_CHANGELIST` generate a diff for each file in /etc/changelist with it's backup

3.5.2 *Modifying Tests*

`CHECK_HOMES_PERMIT_USERGROUPS` allow files to be group-writable if group- equals username

`CHECK_MTREE_FOLLOW_SYMLINKS` let mtree follow symbolic links

`CHECK_PASSWD_NOWARN_SHELLS` Do not warn of shells not listed in /etc/shells

`CHECK_PASSWD_NOWARN_USERS` ignore those users in check_passwd

`CHECK_PASSWD_PERMIT_STAR` ignore those passwords set to "*" in check_passwd

`MAX_GROUPLN` maximum length of groupnames

MAX_LOGINLEN maximum length of usernames

BACKUP_DIR set the backup directory

DIFF_OPTIONS pass this options to diff when generating diffs

PKGDB_DIR set the pkg database directory

BACKUP_USES_RCS uses rcs to keep a repository of backups instead just two copies

3.6 GETTING INFORMATION ABOUT THE SYSTEM

NetBSD offers several utilities to gather information about the system and running processes.

`ps(1)` displays information about running processes. It offers a lot of options to adjust the output format, like `-e` to display the environment, `-L` to list available keywords and so on.

```
$ ps
  PID TT STAT    TIME COMMAND
2068 p1 S    0:04.26 ksh
2884 p1 R+   0:00.00 ps
$
```

Example 10: process status with `ps`

`top(1)` displays information about top CPU processes. The processes are listed in a table which is updated every two seconds. `top` also offers a lot of options to configure the output.

`sysstat(1)` displays system statistics on screen. It has several modules, that can be listed with `"?"` if `sysstat` is active. One module, `vmstat`, shows statistics related to virtual memory usage, process scheduling, device interrupts, system name translation caching, disk I/O and so on. `sysstat -n netstat` will display load average and network connections, as shown by `netstat(1)`

`netstat(1)` displays the status of the network. One can specify eg. the protocol with `-p`, an interface with `-I` or an address family with `-f`.

```
# netstat -n -f inet
```

```

load averages:  0.12,  0.14,  0.10                                22:14:00
48 processes:  47 sleeping, 1 on processor
CPU states:  3.0% user,  0.0% nice,  1.5% system,  0.0% interrupt, 95.5% idle
Memory: 121M Act, 61M Inact, 640K Wired, 8836K Exec, 92M File, 1816K Free
Swap: 263M Total, 132M Used, 131M Free

  PID USERNAME PRI NICE  SIZE  RES STATE   TIME  WCPU   CPU COMMAND
  534 www      18   0 6464K 6396K lockf   1:00  0.24%  0.24% httpd
23646 root       2   0  500K 2168K netio    0:00  0.37%  0.20% sshd
  283 pgsql      2  -2 2716K  676K select  0:32  0.05%  0.05% postgres
15337 stefan    28   0  184K 1028K CPU      0:00  0.00%  0.00% top

```

Figure 3.1.: Output of top(1)

```

  6 users      Load  2.40  2.53  2.38                                Fri Sep  9 22:57:57
Proc:r  d  s  w   Csw  Trp   Sys  Int  Sof  Flt                                PAGING  SWAPPING
  2    26                                3                                in  out  in  out
0.3% Sy  1.1% Us  0.0% Ni  0.0% In  98.6% Id  ops
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
>

      memory totals (in kB)
      real  virtual  free
Active 950308    950308  69964
All    1473376    1473376 843600

Namei          Sys-cache      Proc-cache
  Calls      hits  %      hits  %
    14        14 100

Disks:  md0  wd0  wd1  cd0  cgd0
  seeks
  xfers
  bytes      1691
  %busy

      Interrupts
cpu0 softclock
cpu0 softnet
pic0 pin 14
pic0 pin 15
pic0 pin 9
pic0 pin 10
pic0 pin 5
pic0 pin 1
pic0 pin 12
pic0 pin 6
pic0 pin 0

forks
fkppv
fksvr
pwait
relch
rlkol
noran
ndcpy
fltcp
zfod
cow
64 fmin
85 ftarg
itarg
449 wireo
pdfre
pdscr

```

Figure 3.2.: Output of “vmstat”-module of sysstat(1)

Example 11: network status with netstat

can be used to list all IP connections on the machine. Additionally, several statistics can be displayed either.

MODSTAT(8) lists the status of all loaded kernel modules.

FSTAT(8) lists the status of open files.

NFSSTAT(1) lists NFS statistics.

IPCS(1) lists System V interprocess communication statistics.

PSTAT(8) displays system data structures.

VMSTAT(1) lists virtual memory statistics.

STAT(1) lists file status.

Additionally, there are two packages to monitor network interfaces. bmon [pp15] and slurm [pp14] offer a textbased, but graphical, statistic of network traffic.

[pp14]

[pp15]

SEE ALSO:

man	man
fstat man	ps man
ipcs man	stat man
netstat man	sysstat man
nfsstat man	top man
pstat	vmstat

Chapter 3.7, “System Accounting with sa(8)” on page 45

3.7 SYSTEM ACCOUNTING WITH SA(8)

3.8 KEEPING THE CLOCK IN SYNC WITH NTP

Keeping the accurate time is vital on machines that are used in a distributed network, since the clock is used to synchronize operations within the network. And of course, the timestamp in a logfile should be correct, too.

The Network Time Protocol (NTP) has been defined in [RFC1305](#) and developed to enable synchronized timekeeping among distributed time servers and clients.

NTP organizes timeservers (called “Stratum”) in a hierarchical way at at least three levels. The highest level, Stratum 1, usually synchronizes against an official time emitter like the NIST in Boulder, Colorado or PTB in Braunschweig. These Level 1 server are not available for public acces, but they allow connection only to several accredited Level 2 Stratum server. These Level 2 Server usually are open for public access. One can build an own time sever at Level 3 to feed an own network.

To synchronize a client against a public available NTP Server, one can use `ntpdate(8)`:

```
# /usr/sbin/ntpdate 192.168.1.216
```

Example 12: setting the date with `ntpdate`

This can be automated using `cron` or `/etc/rc.conf` at start up process. `cron` should be used at appropriate intervals if the server is up and running `24 * 7` and the hardware clock is imprecise.

To enable `ntpdate` at start up via `rc.conf`:

```
ntpdate=YES  
ntpdate_flags="-u"  
ntpdate_hosts="192.168.1.216"
```

Example 13: configuring `ntpdate` via `rc.conf`

The desired host can also be configured in `/etc/ntp.conf`

SEE ALSO:

[RFC1305](#)

public NTP servers: <http://ntp.isc.org/bin/view/Servers/WebHome>
[vse](#)

[man](#) [ntpdate](#)

[man](#) [rc.conf](#)

Chapter 3.9, “Schedule Tasks with `cron`, `anacron` or `at`” on page 47

3.9 SCHEDULE TASKS WITH CRON, ANACRON OR AT

To start tasks in a time controlled way, NetBSD offers `cron(8)` and/or `at(1)`.

3.9.1 *cron*

`cron` is some kind of clock timer, it can be used to start programs time controlled. `crontab(1)` is used to maintain the user's cron schedule.

`crontab` uses a time key of seven placeholders:

1. Minute (0 – 59)
2. Hour (0 – 23)
3. Day of Month (0 – 31)
4. Month of Year (1 – 12)
5. Day of Week (0 – 6)

A `"*"` serves as joker, representing all possible values.

`crontab -e` invokes the standard editor (usually `vi`) and opens the cron schedule to be edited:

```
SHELL=/bin/sh
PATH=/bin:/sbin:/usr/bin:/usr/sbin:/usr/pkg/bin/
HOME=/var/log
15 3 * * * ntpdate 192.168.1.216
30 4 * * 6 pg_dump -f 'date +%Y%m%d' users
0/15/30/45 * * * * vacuumdb -all --analyze --full
```

Example 14: `crontab`

The first three lines define environment variables, the fourth starts `ntpdate` each day at 03:45, the fifth starts `pg_dump` at 04:30 Saturdays (0 = Sunday, 1 = Monday) and the last line vacuums PostgreSQL databases every 15 minutes each day.

3.9.2 *anacron*

[pp13]

Anacron is a cron version that also works on machines that are not running around the clock. Anacron executes commands periodically by checking the schedule, where the frequency of command execution is defined.

The package installs `/usr/pkg/share/examples/rc.d/anacron`, which can be used to start anacron via `/etc/rc.conf` for root. Simply copy `/usr/pkg/share/examples/rc.d/anacron` to `/etc/rc.conf.d/` and add `anacron=yes` to `/etc/rc.conf`.

`anacrontab` uses four rows to define a command:

1. `days`: frequency in which the command shall be run (in days)
2. `delay`: minutes to wait before the command is executed
3. `id`: ID that has to be unique for all users
4. `command`: shell command that shall be executed

anacron is controlled by `/usr/pkg/etc/anacrontab`:

```
#days delayid    command
1  5      daily  /bin/sh /etc/daily
7  15     weekly /bin/sh /etc/weekly
```

Example 15: anacron crontab

The first command is started after 5 minutes every day. The second is run every seven days after 15 minutes.

SEE ALSO:

[man](#)
[cron man](#)
[crontab man](#)
[anacron man](#)
[anacrontab](#)

3.9.3 *at and batch*

`at(1)` can be used to start a program at a specified time, `atq(1)` and `atrm(1)` list the pool or remove jobs (like `lpq` and `lprm`).

`batch(1)` starts a program when the time has come and the load level of the machine is below 1.5.

`at` or `batch` accept several forms of a time stamp. The easiest way is the `-t` option, where a time stamp defined as

CCYYMMDDhhmmss

is expected.

CC century, two digits

YY year, two digits

MM month, 01-12, two digits

DD day, 01-31, two digits

HH hour, 00-23, two digits

MM minute, 00-59, two digits

ss seconds, 00-61, two digits

To start a program at Wednesday 12th October 2005, 20:15 use:

```
$ at -t 200510122015
echo 'make backups'
^d
Job 1 will be executed using /bin/sh
$ atq
Date           Owner    Queue  Job
20:15:00 10/12/05 stefan   c      1
$
```

Example 16: scheduling programs with `at`

`at` will invoke a command line, where the job to be run can be specified. To exit `at`, press `^ D` ([Control] + [D]). `atq` lists queued jobs, which can be removed with `atrm`.

batch works the same way, except that jobs are run if the system load is lower than 1.5.

SEE ALSO:

[man](#)

[at man](#)

[batch](#)

3.10 FILE PERMISSIONS

NetBSD as a Unix uses file modes defined in IEEE Std 1003.2-1992 ("POSIX.2") to adjust file access permissions. A file has owner and a group (the creator and his group) and world permissions, where world are all users who are neither the creator nor in his group. The file permissions can be adjusted separately for the owner, group and world.

chmod(1) can be used to change a file's mode. chmod expects at least a mode and a file as option. The mode can be coded either in octal numbers or as symbols. I prefer octal numbers and think they are easier to use, once understood. The modes are combined in a positional system, where 8^0 refers to the world, 8^1 to the group, 8^2 the owner and 8^3 determines some special options.

The options are:

- 1 - (x) execute
- 2 - (w) write
- 4 - (r) read by owner
- 10 - execute
- 20 - write
- 40 - read by group
- 100, 200, 400 - read, write, execute by owner
- 1000 - "sticky bit"
- 2000 - set GID

- 4000 - set UID

These modes can be combined additive, to make an file read/write/executable by the owner, read and executable by the group and readable by the world, one would use: $100 + 200 + 400 + 40 + 10 + 4 = 754$ as file mode which will be listed as `-rwxr-xr-` by `ls -l`.

To change the owner and/or group of a file, one can use `chmod(1)` or `chgrp(1)`. `chmod` can be used to specify user or user and group, `chgrp` sets the group:

```
# chown stefan *
# chown -R stefan.users home/stefan/
# chgrp www /home/www/
```

Example 17: setting file ownership and permissions

SEE ALSO:

[man](#)
[chmod man](#)
[chgrp man](#)
[chown man](#)
[ls](#)

3.11 PROTECTING FILES WITH SECURITY LEVELS AND FILE FLAGS

3.11.1 *Kernel Security Levels*

Kernel security levels have been introduced in 4.4BSD to use file flags as a mechanism to enhance security. Kernel security levels allow the administrator to protect programs and data from being altered even by root. The `init` manpage describes security levels very excessive.

Usually the system is running at a level 1, which can be checked with

```
$ sysctl kern.securelevel
```

Example 18: checking the `kern.securelevel`

once the level has been set in the bootup process using the `securelevel` option in

/etc/rc.conf you cannot lower the level anymore, but you are allowed to raise it to either 1 or 2. In addition to using file flags, a kernel security level greater than 0 will also deny any write-access to kernelmemory /dev/mem and /dev/kmem so it is impossible to manipulate the veriexec signatures (see Chapter 4.1, “[Defending Against Rootkits with Verified Executables](#)” on page 61) loaded into kernelmemory and to manipulate files protected by file flags.

Excerpt from man 8 init:

Security levels are defined as follows:

- 1 Permanently insecure mode - always run system in level 0 mode.
- 0 Insecure mode - immutable and append-only flags may be changed. All devices may be read or written subject to their permissions.
- 1 Secure mode - system immutable and system append-only flags may not be turned off; disks for mounted filesystems, /dev/mem, and /dev/kmem are read-only.

The verified exec in-kernel fingerprint table may not be changed (see veriexecctl(8)).

- 2 Highly secure mode - same as secure mode, plus disks are always read-only whether mounted or not, new disks may not be mounted, and existing mounts may only be downgraded from read-write to read-only. This level precludes tampering with filesystems by unmounting them, but also inhibits running newfs(8) while the system is multi-user.

The settimeofday(2) system call can only advance the time.

The state of ipf(8) (the in-kernel IP filtering facility) may not be changed.

Users may not change the per-process core name template format, only the default can be changed.

3.11.2 *Protecting Files with File Flags*

File flags are a way to protect files by forbidding (or allowing) different operations on files. A file flag can be set with the chflags command.

Available flags are:

ARCH † 'archived' — the file is archived

OPAQUE set the opaque flag

NODUMP 'nodump' — the file won't get dumped with `dump(8)`⁴

SAPPND † 'system append' — the file is set to append-only

SCHG † 'system changeable' — the file is not changeable

UAPPND 'user append' — the file is set to append-only

UCHG 'user changeable' — the file is not changeable

† : only usable by root

SEE ALSO:

`man init`

`man chflags`

`/usr/include/sys/system.h`

`ff`

3.12 FILE SYSTEM EXTENDED ATTRIBUTES (*netbsd 3.0*)

"Extended attribute support was developed as part of the TrustedBSD Project, and introduced in FreeBSD 5.0 and NetBSD 3.0. It was developed to support security extensions requiring additional labels to be associated with each file or directory." - from `man extattr`

Extended Attributes allow one to add meta data to vnodes of files and directories. This can be used to keep user defined information (eg. a checksum) connected to a file/directory. The attributes are arranged within several namespaces⁵, where two names exist by default: `EXTATTR_NAMESPACE_SYSTEM` and `EXTATTR_NAMESPACE_USER`. The system namespace can only be set with appropriate system privileges⁶, while the user namespace is protected with the file's permissions.

⁴ for level 0 only if `-h` is passed

⁵ they depend on the underlying file system

⁶ What the fuck does this mean?

SEE ALSO:

gettextattr, lsextextattr, rmextattr, settextattr

3.13 LOGGING SYSTEM MESSAGES WITH SYSLOG

The syslog-daemon `syslogd` is a facility to log system messages to a file, console or another system.

It is important for an administrator to enable syslogs and to read and interpret them, since they are a superb source of information about the system's activities.

The `syslogd` should be started at boottime via `/etc/rc.conf` with the options `syslogd=yes` and `syslogd_flags` to pass further options to `syslogd`. Additionally, several other serverprocesses like `inetd` or `ipmon` can be commanded to log messages to `syslogd`.

`syslogd` is configured via `/etc/syslog.conf`

Another interesting utility might be `logger`, it allows one to pass a message to `syslogd` like that:

```
$ logger 'need to reboot to adjust file flags'
```

Example 19: logging to `syslogd` with `logger`

3.14 CHECK FOR INSECURE PACKAGES WITH AUDIT-PACKAGES

The easiest way to install software on NetBSD⁷ is `pkgsrc`⁸. In one sentence, `pkgsrc` is a collection of Makefiles required to build a package and its dependencies. One can build a complete release of TeX_Ε by checking out `pkgsrc` via CVS (or downloading a tarball), going to `/usr/pkgsrc/print/tex` and entering `make install clean`. `Pkgsrc` will automatically resolve TeX_Ε's dependencies, download, compile and install them. To maintain installed packages, `pkgsrc` uses a simple database. The database can be used to list all packages with `pkg_info | sort`. Additionally, one is able to check if the installed system uses exploited software with a simple program called `audit-packages`. It can be installed and used with:

⁷ `pkgsrc` is nowadays a project of its own and runs on several other OS, too

⁸ package source is the full name

```
$ cd /usr/pkgsrc/security/audit-packages/  
$ make install clean  
$ download-vulnerability-list && audit-packages
```

Example 20: installing audit-packages

The program will download the vulnerability list (a plain text file listing security issues) and compare the installed packages against that list. `audit-packages` will list all installed exploited packages like that: Package `gaim-1.2.0` has a denial-of-service vulnerability, see <http://gaim.sourceforge.net/security/?id=13>, so the administrator is able to replace them by newer versions using `pkgsrc`.

It is a good idea to automate this task with a cronjob, that will send the output via eMail.

More information on `pkgsrc` can be found at <http://www.pkgsrc.org>

3.15 SYTEM CALL POLICIES WITH SYSTRACE

3.16 CHANGING THE ROOT DIRECTORY WITH CHROOT

3.17 MAKING BACKUPS WITH DUMP

A backup is a copy of important data in case the original source is damaged. These damage can be caused by hardware failure (head crash of a harddrive, surge etc.), user failure (`rm -rf /`) and of course by an intruder, so it is vital to have copies of important data.

Making backups is a very complex section, especially in networks on distributed systems and/or systems that use complex software like CAD/CAE or databases, so I'll focus on personal workstations.

3.17.1 *Organising the Backup*

First one has to discover which data has to be backed up, usually this is `/home/` and data from applications, like the document root of Apache, a dump of PostgreSQL databases, CVS root and mail spools.

Second it is vital to determine a backup strategy - how often shall the data be backed up? Is a monthly basis enough or does the data change each day, so daily backups are required?

The third question is the way how backups should be made. There are three different ways:

- complete backup - all data is saved, each time
- differential backup - only data that has been changed since the last complete backup is saved
- incremental backup - all data is saved that has been changed since the last backup

A complete backup is the easiest - simply dump all required directories to a tape or CD. Restoring the data goes the same way, simply restore the archive. But there is a critical factor - time and space used. Each backup will copy all data - even if it didn't change. This is suboptimal eg. if you have several hundred MBs of digital photos that do not change - but are dumped every backup.

To avoid that problem, one can use a so called differential⁹ backup. A differential backup saves only data changed since the last full backup. This saves time and space, but it is a little bit more complicated, especially regarding restore. An appropriate strategy could be to backup all data on monday and perform a differential backup the other weekdays. If the system has to be restored on a thursday, one has to restore the monday tape full and then additionally the last available tape from wednesday.

A little bit more complicated but even more time and space saving is an incremental backup. Each backup saves only the data changed since the last backup. So the following strategy could be established: a full backup at the first of the month and an incremental backup each other day. Performing a restore at 16th of the month would require one to restore the tapes from the first up to the 16th in chronological order - which is very inefficient. I use a mixed strategy composed of all three ways: I perform a complete (level 0) backup the monday of the month, and a differential (level 1) backup on each other monday of the month. The other days, incremental backups using level 2 are made.

So an example plan for September 2005¹⁰ is shown in the following table. The superscript numbers are the dump levels.

⁹ sometimes named delta, after the math δ

¹⁰ use `cal 9 2005` to display it on the terminal

M	Tu	W	Th	F	S	S
29 ¹	30 ²	31 ²	1 ²	2 ²	3 ²	4 ²
5 ⁰	6 ²	7 ²	8 ²	9 ²	10 ²	11 ²
12 ¹	13 ²	14 ²	15 ²	16 ²	17 ²	18 ²
19 ¹	20 ²	21 ²	22 ²	23 ²	24 ²	25 ²
26 ¹	27 ²	28 ²	29 ²	30 ²	1 ²	2 ²

01.09. - 04.09. will be handled by August tapes, as 01.10. and 02.10 will be included into the September cycle.

After a crash at Thursday, 22.09. the restore order would be:

1. tape level 0 from Mon. 05.09.
2. tape level 1 from Mon. 19.09.
3. tape level 2 from Wed. 21.09.

Another important issue are the backup media. Traditionally, magnetic tapes have been used and are still in use among professional users today. They are available as cabinet sized jukeboxes with robot arms to change tapes with several hundred GBs of size or as simple streamer for a 5.25 inch slot of a standard pc with some GB capacity. Modern streamers are fast, *very* reliable and can be bought for a low price at online marketplaces.

Of course, one can use other removable media, too, like ZIP Diskettes, USB Sticks or plain vanilla CD-RW/DVD-RW. It depends on the size of the archives and how often backups are made. Burning a DVD each day could be a little expensive, esp. if the media fails.

However, it depends on the available hardware (if there is a DVD-RW drive, one can use it) and how much money can be spend. The important items are reliability, availability (What if the drive burned out? Can you still access the media?) and of course size and speed (Would you like to dump 1GB to 3.5 inch floppys?).

It is a very good idea to archive backup media and store them in a safe place (fire proof safe or safe deposit box). Even the best backup media may be corrupted or the backedup data can be corrupted. Sometimes, a file system crash occurred and destroyed data unrecognized or a virus or intruder manipulated data undetected. If one only has backups made after the corruption, the data is gone, but if you have an archived version before the corruption begun, you can restore that. It is a good strategy to use a fresh set of tapes per month and either two different tapes for each archive or a fresh one for each day. If one uses one single tape for the whole month,

and the tape is destroyed by a broken streamer, the whole month is lost (eg. a crash on 22.09. would require one to use the tape from 04.09.).

3.17.2 Using *dump* and *restore*

As already mentioned above, *dump* uses ten levels (0-9) to manage differential backups. A level 0 *dump* dumps all data of a filesystem and even ignores the *nodump* file flag unless *-h* is specified. A level greater than 0 dumps all data since the last backup performed with current level – 1 was performed. An example strategy is described in the calendar above.

dump dumps per default to the rewinding tape device */dev/rst0* but with the *-f* switch one can define another device or file.

```
1 # mount_mfs -s 300m swap /ramdisk && cd /ramdisk
2 # dump -0au -h0 -f- /home/ | bzip2 \
  > home-`date +%y%m%d`.bz2
3 # dump -0auf- / | bzip2 | ssh stefan@192.168.2.1 \
  dd of=/home/stefan/file.bz2
```

Example 21: creating a ramdisk and running backups

The above script creates a memory file system (also known as ramdisk), enters it and dumps */home* at level 0 with auto sized tape length, updating */etc/dumpdates*, honoring the *NODUMP* file flag. The dump is piped through to *bzip2* and written as *home-050904.bz2*. The archive can now be dumped to a streamer or copied to a hard drive. Command 3 dumps */*, pipes it through *bzip2* and pipes it to *ssh* that writes the file via *dd* to */home/stefan/file.bz2* on host 192.168.2.1 as user *stefan*. If *ssh* key authentication (see Chapter 5.3.3, “Using OpenSSH” on page 81) is enabled, no password is required.

I create checksums of the backedup files with *mtree* and checksums of the archive with *sha1* to check if a file might have been corrupted. And of course, I encrypt my backup archives with *mcrypt*, before writing them to tape. By using a ramdisk, I do not have to wipe the unencrypted archive, otherwise, one should use a wiper to secure delete unencrypted backups.

To restore a dump, *restore* can be used, either in interactive mode, where files can be selected, or in recursive mode, where the whole dump is restored. According to the example calendar of September 2005 used above, one would use:

```
0 # cd /home/  
1 # bunzip home-050905.bz2 && restore -r -f home-050905  
2 # bunzip home-051905.bz2 && restore -r -f home-050905  
3 # bunzip home-052105.bz2 && restore -r -f home-052105  
4 # rm restoresymtable
```

Example 22: restoring backups with restore

SEE ALSO:

man dump

man restore

[backup](#)

[zwick1](#)

[zwick2](#)

[gutman](#)

Chapter [ff](#), “[Bibliography](#)” on page [132](#)

Chapter [4.2](#), “[Checking Filesystem Integrity with mtree\(8\)](#)” on page [63](#)

Chapter [6.3](#), “[Symmetric Encryption with mcrpyt](#)” on page [99](#)

Chapter [6.5](#), “[Deleting Data the Secure Way \(“wiping”\)](#)” on page [100](#)

3.18 REDUNDANT DISKS WITH RAID

3.18.1 *Hardware RAID*

3.18.2 *Software RAID with Vinum*

“VIREN SIND DAS TSCHERNOBYL
DER DATENVERARBEITUNG.”
Wau Holland

4.1 DEFENDING AGAINST ROOTKITS WITH VERIFIED EXECUTABLES

veriexec adds a new function to the exec-path of the kernel, thus allowing the kernel to check a cryptographic hash for a binary. With this feature, it is almost impossible to run manipulated binaries like a or a trojan.

4.1.1 How to Enable it

If you want to use veriexec, you have to build a release with veriexec supported, as described below:

Either you use the GENERIC_VERIEXEC-Kernel or you add

```
options VERIFIED_EXEC
options VERIFIED_EXEC_FP_MD5
options VERIFIED_EXEC_FP_SHA1
options VERIFIED_EXEC_FP_RMD160
options VERIFIED_EXEC_FP_SHA512
options VERIFIED_EXEC_FP_SHA384
options VERIFIED_EXEC_FP_SHA256
pseudo-device veriexec 1
```

Example 23: configuring the kernel for VERIEXEC

to your kernel configuration and recompile a new kernel and userland, as described in ['Building the system']guide

If you boot into the new kernel with `veriexec` enabled, you will receive warning messages about inappropriate checksums, ignore them until your userland has been setup to support `veriexec` properly.

After installing the new userland, you are required to create `/dev/veriexec` with

```
$ cd /dev && ./MAKEDEV veriexec
```

Example 24: creating the `veriexec` devices

If done so, you should now create a database containing the system fingerprint, using `/usr/share/examples/veriexecctl/gen\_sha1` as a helper script, you can adjust it to your needs (eg. required path).

The system will generate a file called `signatures`, containing all files and fingerprints. It is a good idea to move `signatures` to a write-protected medium, like a floppy or to encrypt or sign it with e.g. GnuPG, to enable you to check it's integrity.¹

Copy `signatures` to `/etc/` and add `veriexecctl /etc/signatures to /etc/rc.local` to load the signatures into kernelmemory.

If you reboot now and raise `kern.securelevel` to 1, the kernel warns of not matching fingerprints for binaries, if you raise the level to 2 the kernel will refuse to execute binaries with nonmatching fingerprints.

Since you are required to use a `kern.securelevel` greater than -1, X won't run any longer on your machine, since it uses memory mapping to `/dev/mem` to acces your videocard. There exists a workaround called [sysutils/aperture](#), which allows mapping of locked mem areas to the Kernel.

SEE ALSO:

[man](#) [init](#)
[vepaper](#)

¹ on my Webserver, I created an OpenSSL Key and signed `/etc/signatures`, the signature is checked each night via cron to ensure the database hasn't been altered

4.2 CHECKING FILESYSTEM INTEGRITY WITH `MTREE(8)`4.2.1 *What is mtree?*

`mtree` is a utility to compare a directory hierarchy to a file specifying the attributes of that hierarchy. It is usually used to check if a update from binary sets or source code has been succesfully finished. But it is also a very useful tool to enhance the security of a host. It can be used somewhat like `verified_executables` (see Chapter 4.1) to check the integrity of a filesystem. Once your system has been installed and configured, it is a good idea to create a fingerprint of it. This fingerprint should be protected from manipulation with a cryptographic signature or by copying it to a read only medium.

The running system can be compared against the initial fingerprint, so `mtree` will list all files that are either missing in the filesystem or the specification, or that do not match the specification.

This will give a good summary on the system's status and allows one to find out easily if files have been altered eg. by intruders or root kits.

`mtree` uses several keywords to determine the fileattributes that shall be written to the fingerprint:

`IGNORE /path/` ignore everything beneath `/path/`

`GID,GNAME,UID,UNAME` list group ID, Name, User ID, Name

`MD5, RMD160, SHA1` generate checksum of the given type

`MODE` octale filemode

`SIZE` filesize in b

`TIME` timestamp of last modification

These options determine the way `mtree` traverses the file hierarchy:

`-P /PATH/STARTPOINT/` start at given path

`-X FILE` files listed in `File` (relative to `/path/startpoint/`) shall be excluded

`-L` follow symlinks

4.2.2 Example Usage

```
$ mtree -L -c -K sha1,rmid160,gname,uname,mode -p / \  
-X ~/mtree.excl > ~/mtree.orig
```

Example 25: generate a filesystem fingerprint with mtree

will generate a fingerprint starting in / and ignoring all paths (eg. /home/, /tmp/) listed in /etc/mtree.excl.

The fingerprint will look like this:

```
# ./bin  
/set type=file uname=root gname=wheel mode=0555 nlink=1  
  flags=none  
bin   type=dir mode=0755 nlink=2 time=1114894264.0  
  [   nlink=2 size=9975 time=1114894264.0  
      rmid160=1c674146b57bdfb1e13767f031e3904b4db9ec99  
      sha1=889c4cec9477fecef67636cfcac1a2c6c402cc86  
cat   size=10550 time=1114894254.0  
      rmid160=fe5ae34f164466a1203d3950bd2e005780911b08  
      sha1=5d666fa34dc05918870156a34c829fd5e392a557  
chio  size=13730 time=1114894254.0  
      rmid160=cb910729c1038ec5304079e76e6d30b7d42dd918  
      sha1=7502d59bfda502db2ec1bc529c7cd91270b949b2
```

Example 26: example mtree fingerprint

to compare the running system with the initial fingerprint, use:

```
$ mtree -L -p / -X ~/mtree.excl -f ~/mtree.orig
```

Example 27: comparing a mtree fingerprint

Once again: ensure /root/mtree.orig hasn't been altered by an intruder! Use a cryptographic signature made with OpenSSL or GnuPG and keep the private key away from the host, or copy mtree.orig to a readonly medium.

mtree will exit with "0" if no difference has been found and with "2" if the fingerprint didn't match the system, additionally the differences will be listed:


```
15.txt: modification time (Wed Jun  1 17:38:43 2005,  
                        Wed Jun  1 17:41:25 2005)  
      rmd160 (0x228c27c3175774fa0f17fcb3ebc14cb0790e57bf,  
             0x4961df2b594ebfa03d0259c3bebf6c73f22f6709)  
      sha1 (0x5c75c5c01bfe0609bdcdb33c385f00ae1c359d7f,  
           0xaf89b7361dd7f7c7504729545d0a90a92ef37830)  
16.txt: modification time (Wed Jun  1 17:38:43 2005,  
                        Wed Jun  1 17:41:25 2005)  
      rmd160 (0x228c27c3175774fa0f17fcb3ebc14cb0790e57bf,  
             0x4961df2b594ebfa03d0259c3bebf6c73f22f6709)  
      sha1 (0x5c75c5c01bfe0609bdcdb33c385f00ae1c359d7f,  
           0xaf89b7361dd7f7c7504729545d0a90a92ef37830)  
extra: newfile.txt  
missing: ./17.txt
```

Example 28: mtree report

The files 15.txt and 16.txt have been altered, 17.txt has been deleted (it's missing in the system) and newfile.txt was created on the system, but isn't listed in the fingerprint.

4.3 CHECKING FILESYSTEM INTEGRITY WITH AIDE

4.3.1 *What is AIDE?*

[pp6] AIDE (Advanced Intrusion Detection Environment) is a free file integrity checker, therefore it creates a database from the regular expression rules that it finds from the config file. Once this database is initialized it can be used to verify the integrity of the files. It has several message digest algorithms (md5, sha1, rmd160, tiger, haval, etc.) that are used to check the integrity of the file. All of the common file attributes can also be checked for inconsistencies. This allows you to check if files have been manipulated for example by a worm, virus or intruder (e. g. a rootkit or trojaned program has been installed).

4.3.2 *Configuring AIDE*

After installation finished you have to configure AIDE and create a database, containing the data for each file chosen to be checked. The default path to the

configuration file is `/usr/pkg/etc/aide.conf`, but this can be altered using the option `-config=configfile`. The configuration file is easy to edit, it contains the directories to check and the options which should be used to check them.

```
database = file:///etc/aide.db
# path to AIDE database in URL format

database_new = file:///root/aide.db.new
# path to new AIDE database which shall be created

report_url = file:///root/aide.report
# path to generated report
```

Example 29: aide.conf

available options of AIDE are:

- p : permissions
- n : number of links
- u : user
- g : group
- s : size
- b : block count
- m : mtime
- c : ctime
- a : atime
- i : inode
- S : check for growing size
- E : empty group
- md5 : MD5 checksum

- sha1 : SHA1 checksum
- rmd160 : RMD160 checksum
- tiger : TIGER checksum
- R : p+n+u+g+i+s+m+c+md5
- L : p+n+u+g+i
- > : growing logfile and p+g+u+n+i+S

These options are almost self-explaining, each submitted option will check in and compare the regarding fileattribute. "R", "L" and ">" are just macros for the corresponding options, so you can also build your own macro in the same way, e.g.:

```
daemon  : md5+sha1+rmd160+tiger
heavy   : sha1+p+u+g+m+c+a+i+n
```

Example 30: AIDE makro

'daemon' will create and check MD5, SHA1, RMD160 and TIGER checksums.

Now it's time to configure the files to be checked:

```
/etc          heavy
/bin          heavy
/sbin         heavy
/usr/bin      heavy
/usr/sbin     heavy
/usr/pkg/bin  heavy
/usr/pkg/sbin heavy
# check with heavy-settings

/root         md5
/dev          u+g+i+s+S
#check with specified options

!/var/spool/
!/home/students
# do not check below this directories
```

Example 31: AIDE configuration

This also done very easily, just list each directory and the options which should apply to it. Directories that should not be checked e. g. because they change very often, like `/var/spool`, or privacy of users might be violated, can be excluded with a single bang (!).

4.3.3 Example Usage

After AIDEs configuration is ready, you can start to build an initial database with:

```
aide -i
```

Example 32: initializing AIDE

The created database is usually stored in `/usr/pkg/etc/`, except you specified another path in `aide.conf`. Since this database is the fingerprint each other AIDE comparison run uses to compare with, you should ensure its integrity by protecting it. This can be done by cryptography (e.g. encrypt or sign the database with PGP/GnuPG) or simply moving it to a protected media like CDROM or write protected floppy / ZIP disk. If you are paranoid², you can also move AIDEs binary onto that media and deinstall the package to deceive an intruder. Now you can run AIDE to check the status of your filesystem, this is simply done by

```
# aide -C
```

Example 33: checking a filesystem with AIDE

AIDE now builds a new database with the current status of all files and compares it against the initial database. Of course, AIDE needs permission to open all needed files, so it must be started as root to check system files.

```
/etc/ld.so.conf 4217405 100644 0 0 55 156666 1  
56ALmHB6PCPZN8bkbYHXvg== R+47jcSQFV59spXxQip+VJd+Jdw=  
/etc/ttyaction 4217405 100644 0 0 123 156667 1  
wHSegaYfbPPg1MbNbvSV0Q== kufkrktfUPSYfMp4zyym2M7pyJE8=  
/etc/motd 4217405 100664 0 0 1551 156668 1  
PQ27jGbXvZUXb+Jsi7uNkg== X82o0Zx8XV47nA9zF939H7aHc+A=  
/etc/hosts.equiv 4217405 100600 0 0 2 156669 1  
Ch8ho0FzieDAoT0Sx5p6iQ== 4Axzy+iVsOK5B7wmqjQs0xAKgEs=
```

² You are, otherwise you wouldn't read this book ;-)

Example 34: excerpt from a AIDE database

Now, `/etc/motd` has been modified and `aide -c` has been run, excerpt from the generated report:

```
changed:/etc/motd
Detailed information about changes:

File: /etc/motd
Size      : 1551              , 1547
Inode     : 156668           , 156819
MD5       : PQ27jGbXvZUXb+Jsi7uNkg==
           , BPOFEUF6A8tt0xkeHnT2ow==
RMD160    : X82o0Zx8XV47nA9zF939H7aHc+A=
           , S6ZmfxfjV0SYPPoBrt5TS3Z7fn00=
```

Example 35: AIDE report

4.4 SCANNING FOR VIRUSES

4.4.1 Mail Virus Scanner *amavis*

4.4.2 F-prot Antivirus for Home Users

4.4.3 Mail Content Filtering with *avcheck*

4.4.4 ClamAV

4.5 SCAN YOUR MACHINE FOR ROOTKITS WITH CHKROOTKIT

`chkrootkit` is a toolset to find signs of rootkits on your machine, it is available in `pkgsrc`. It works somewhat like a virus scanner, because it scans system binaries for rootkit modification, additionally, it contains of further programs:

[pp2]

- `chkrootkit`: a shell script that checks system binaries for rootkit modification.
- `ifpromisc.c`: checks if the network interface is in promiscuous mode.

- `chklastlog.c`: checks for lastlog deletions.
- `chkwtmp.c`: checks for wtmp deletions.
- `check_wtmpx.c`: checks for wtmpx deletions. (Solaris only)
- `chkproc.c`: checks for signs of LKM trojans.
- `chkdirs.c`: checks for signs of LKM trojans.
- `strings.c`: quick and dirty strings replacement.
- `chkutmp.c`: checks for utmp deletions.

`chkrootkit` can be run with:

```
# chkrootkit
```

Example 36: running `chkrootkit`

It will perform all tests for the system binaries. The option `-l` lists all available tests, which can be selected separately, `-r DIR` uses `DIR` as the rootdirectory of the tests, eg. if you have a suspicious systemdisk mounted into a fresh system, `-p DIR` sets the dir for binaries required by `chkrootkit`, eg. `ps`, `grep` or `cut`. This is useful if you have `chkrootkit` on a CD, to protect it from being altered. `-x` will show suspicious data in binaries, so the user can examine if a rootkit has been found.

A rootkit might be designed to attack `chkrootkit`, preventing it from being discovered, so it is a good idea to check a suspicious system from a known clean system, like a CD-based live system.

Since `chkrootkit` uses pattern matching to find rootkits, it can only find known rootkits. Even if `chkrootkit` finds no malware, a rootkit might be installed, though. A list of known rootkits can be found at <http://www.chkrootkit.org/#tests>.

4.6 SCANNING THE NETWORK FOR ZOMBIES (DDOS CLIENTS)

[pp19]

`ddos-scan` is a package [pp19] that scans a network for so called zombies (trino / TFN / stacheldraht agents). Zombies are computers or program that are infected with dDoS-clients, which can be controlled by a master. These machines can be used in a distributed Denial of Service attack to make websites or other services unavailable by exhausting the system's resources.

All those zombies do not affect NetBSD, but a widespread proprietary group of operating system. If there are some of those “systems” in your network, it is a good idea to scan them.

```
# dds -v -d 127.0.0.1
No mask passed, assuming host scan (/32)
Mask: 32
Target: 127.0.0.1
dds $Revision: 1.132 $ - scanning...

Probing address 127.0.0.1
Sending ICMP to: 127.0.0.1
Sending tfn ICMP to: 127.0.0.1
Sending UDP to: 127.0.0.1
Listener got an ICMP packet
Waiting 30 seconds for late responses.
Listener got an ICMP packet
Listener got an ICMP packet
#
```

Example 37: scanning the net with ddos-scan

SEE ALSO:

[cert9907](#)

[ca200001](#)

“DER MENSCH IST EIN SEIL,
GEKNÜPFT ZWISCHEN TIER UND ÜBERMENSCH,
EIN SEIL ÜBER EINEM ABGRUNDE.”
Friedrich Nietzsche

5.1 THE INTERNET PACKET FILTER IPF

5.2 THE INTERNET SUPERSERVER INETD

5.3 ENCRYPTED LOGINS AND NETWORK CONNECTIONS WITH OPENSSSH

OpenSSH is installed in the NetBSD base distribution and offers an replacement for telnet, ftp, rcp and rsh using strong cryptography.

5.3.1 *Configuring the Server*

AFSTOKENPASSING *Default: Yes* determines if an AFS Token shall be forwarded

ALLOWGROUPS

Allow only specified groups. If used, all non specified groups are blocked.
Jokers like “?” and “*” can be used.

ALLOWTCPFORWARDING

Allow TCP-Forwarding

ALLOWUSERS

Allow only specified users. If used, all non specified users are blocked. Jokers like “?” and “*” can be used.

AUTHORIZEDKEYSFILE *default: .ssh/authorized_keys*
defines the public key file of a user, the following jokers can be used: %% = %, %h = home of the user, %u = username

BANNER *default: off, only in v2*
determines a file that is displayed *before* the login. Nur in Protokoll 2, Standard abgeschaltet

CHALLENGERESPONSEAUTHENTICATION *default : yes*
enables challenge response authentication, all authentication methods of login.conf(5) are supported

CIPHERS *default: aes128-cbc,3des-cbc,blowfish-cbc,cast128-cbc,arcfour, aes192-cbc,aes256-cbc, regards v2*
defines ciphers to be used in v2

CLIENTALIVEINTERVAL *v2 only*
the daemon checks the clients activity and availability, 0 equals no check. see also **ClientAliveCountMax**

CLIENTALIVECOUNTMAX *default: 3*
count of failed **ClientAliveInterval** checks, before the client will be timed out, the messages are transmitted within the encrypted channel

DENYGROUPS *default: none*
deny specified groups, Jokers are allowed, GIDs not

DENYUSERS *default: none*
deny specified users, Jokers are allowed, GIDs not, “user@host” may be used to deny a user from a special host

PUBKEYAUTHENTICATION *default=yes, v2 only*
enable public key authentication

GATEWAYPORTS *default: no*
Bind forwarded ports to any address used by sshd, instead localhost only. This allows other hosts to connect to the forwarded ports.

HOSTKEY *parameter: -h*
defines the private key file(s) of the daemon, files with group/world access are ignored.

- IGNORERHOSTS** *default: yes*
ignore `.rhosts` and `.shosts`, `/etc/hosts.equiv` and `/etc/shosts.equiv` will be interpreted, though
- IGNOREUSERKNOWNHOSTS** *default: no*
if activated, ignores `$HOME/.ssh/known_hosts` during
RhostsRSAAuthentication
- KEEPALIVE** *default: yes, client and server have to disable it*
sends packets to check if the client is still connected, if the connection breaks, the session will be destroyed
- KERBEROSAUTHENTICATION** *default: yes*
defines if authentication via Kerberos is allowed
- KERBEROSORLOCALPASSWD** *default: yes*
authentication falls back to password, if Kerberos fails
- KERBEROSTGTPASSING** *default: no*
defines if a Kerberos TGT shall be passed to the daemon
- KERBEROSTICKETCLEANUP** *default: yes*
deletes Kerberos ticket at logout
- KEYREGENERATIONINTERVAL** *parameter -k, default: 3600s, v1 only*
The private server key is regenerated after the specified seconds. This is done to prevent an intruder from stealing a key to decrypt captured sessions.
- LISTENADDRESS** *default: all*
Local addresses `sshd` should listen on.
- LOGINGRACETIME** *parameter: -g, default: 120s*
The server disconnects a session after `n` seconds, if no login succeeded.
- LOGLEVEL** *default: info*
Verbose level of `sshd`.
- MACS** *default: hmac-md5,hmac-sha1,hmac-ripemd160,hmac-sha1-96,hmac-md5-96*
Define available MACs
- MAXSTARTUPS** *default: 10*
Maximum number of concurring unauthenticated connections to the `sshd`.
More will be dropped
- PASSWORDAUTHENTICATION** *default: yes*

Allow authentication with password

PERMITEMPTYPASSWORDS *default: no*
Allow empty passwords.

PERMITROOTLOGIN *default: no*
Allow or disallow login of root.

PIDFILE
i an, die die Prozess ID (pid) des sshd enthaelt. Standard ist /var/run/sshd.pid

PORT Parameter: -p
Gibt den Port an, an den sich der sshd binden soll. Standard ist 22, mehrere Ports koennen uebergeben werden.

PRINTMOTD
Gibt an ob sshd beim Login /etc/motd ausgeben soll. Wird auf manchen System bereits von der Shell uebernommen Standard ist ja.

PROTOCOL
Gibt die Protokollversionen an, Standard ist 1, 1,2 erlaubt 1 und 2

PUBKEYAUTHENTICATION
Erlaubt das Login per Schluessel.

RANDOMSEED
Veraltet, Zufallsgeneratoren benutzen andere Techniken.

REVERSE MappingCHECK
sshd ueberprueft ob der Clienthostname auf die uebertragene IP passt.
Standard ist nein.

RHOSTSAUTHENTICATION
Erlaubt rhosts-Authentifizierung mittels .rhosts oder /etc/hosts.equiv Standard ist nein.

RHOSTSRSAAUTHENTICATION
Erlaubt rhosts-Authentifizierung mittels .rhosts oder /etc/hosts.equiv und RSA Host-Authentifizierung Standard ist nein.

RSAAUTHENTICATION
Gibt an das reine RSA Authentifizierung erlaubt ist. Standard ist ja, betrifft nur Protokoll 1.

SERVERKEYBITS

Gibt die Groesse der Serverschlüssels in Bit an. Minimum ist 512, Standard ist 768

STRICTMODES

Gibt an ob sshd Owner und Modes der Dateien checken soll. Anfänger lassen ihre Dateien manchmal World-writable, Standard ist ja.

SUBSYSTEM

Spezifiziert ein externes Subsystem (z. B. Secure FTP). Optionen sind ein Subsystem Name und der Befehl der bei einem Request ausgeführt werden soll. Beispiel sftp: Subsystem sftp /usr/libexec/sftp-server

SYSLOGFACILITY

uebergibt den facility code an syslogd, Optionen sind DAEMON, USER, AUTH, LOCAL0, LOCAL1, LOCAL2, LOCAL3, LOCAL4, LOCAL5, LOCAL6, LOCAL7 Standard ist AUTH.

USELOGIN

Login(1) soll fuer interaktive Logins benutzt werden. Login wird nicht fuer remote Befehle verwendet. Standard ist Nein

X11DISPLAYOFFSET

Spezifiziert die erste Displaynummer fuer X11 forwarding, um Probleme mit echten X11 Servern zu vermeiden. Standard ist 10.

X11FORWARDING

Gibt an ob X11 Forwarding erlaubt werden soll. Standard ist nein. Das Verboten erhoehrt die Sicherheit nicht da die User eigene Forwarder installieren koennen.

XAUTHLOCATION

Gibt den Ort des xauth(1) Programm. Standard ist /usr/X11R6/bin/xauth.

5.3.2 *Configuring the Client*

The OpenSSH client program ssh gets the configuration either from: command line, \$HOME/.ssh/config or finally /etc/ssh.conf

Parameters of ssh are:

HOST

Begrenzt bis zu naechsten "Host" die Regeln auf den uebergebenen Host.

Jokerzeichen * und ? sind erlaubt

AFSTOKENPASSING

→ sshd

BATCHMODE

use batch mode, no password is asked for if pub key auth is enabled, useful in scripts

CHECKHOSTIP

compare IP address with the one in `known_hosts` to detect spoofed connections with the key fingerprint

CIPHER *v1*

cipher to be used in *v1*

CIPHERS *v2, default: aes128-cbc, 3des-cbc, blowfish-cbc, cast128-cbc, arcfour, aes192-cbc, aes256-cbc*

ciphers to be used in *v2*

COMPRESSION *default: no*

enables compression

COMPRESSIONLEVEL *v1, default: 6*

compression level, like gzip, ranges from 0 (fast) up to 9 (best compression, slow)

CONNECTIONATTEMPTS *default: 1*

number of connection attempts before the connection will be canceled, useful in scripts

PUBKEYAUTHENTICATION *v2*

enables authentication via public key

ESCAPECHAR *default: ~*

defines escape character, useful for binary data

FORWARDAGENT *default: no*

forward the connection to the auth agent

FORWARDX11 *default: no*

sets `$DISPLAY` and forwards X11 connections through the encrypted channel

GATEWAYPORTS *default: no*

defines if remote hosts can connect to local forwarded ports

GLOBALKNOWNHOSTSFILE

use another file instead `/etc/ssh_known_hosts` for host keys

HOSTKEYALIAS

Use this alias instead the one defined in `known_hosts`, useful for tunneling or to differentiate multiple daemons on a single machine

HOSTNAME

defines an alias of a host to be written into the logs

IDENTITYFILE *default: \$HOME/.ssh/identity*

file(s) that contains the user's RSA/DSA identity

KEEPALIVE *default: yes*

defines if "keep alive" messages should be sent, kills the session if the connection breaks

KERBEROSAUTHENTICATION

Spezifiziert ob Kerberos-Authentifizierung erlaubt ist. Standard ist ja, der Server benoetigt eine Kerberos Servtab welche die Authentifizierung der KDC Identitaet erlaubt. Die Identifizierung erfolgt mittels Kerberos Ticket oder Passwort.

KERBEROSTGTPASSING

Gibt an ob ein Kerberos TGT an den Server geforwardet werden soll. Standard ist nein, es funktioniert nur wenn die Kerberos KDC ein AFS-Kaserver ist.

LOCALFORWARD

Definiert einen TCP/IP Port der von der lokalen zur entfernten Maschine geforwardet wird. Mehrere Ports koennen geforwardet werden, Format ist: `lokalerPort:entfernteMaschine:entfernterPort`, nur root kann privilegierte Ports forwarden.

LOGLEVEL

Verbosemodus ("Auskunftsfreudigkeit") dess sshd wenn mitgeloggt wird. Optionen: QUIET, FATAL, ERROR, INFO, VERBOSE and DEBUG. Standard ist INFO. DEBUG verletzt die Privatsphaere der User und ist nicht empfohlen.

MACS

Gibt die MACs (Message Authentication Code) Algorithmen an. Dies wird in Protokoll 2 benutzt um die Integritaet der uebertragung zu schuetzen. Mehrere Algorithmen koennen per Komma-getrennter Liste angegeben werden. Standard ist: `"hmac-sha1,hmac-md5,hmac-ripemd160,hmac-ripemd160@openssh.com,hmac-sha1-96,hmac-md5-96"`

NUMBEROFPASSWORDPROMPTS

Anzahl der Passwortabfragen vor Abbruch. Standard ist 3.

PASSWORDAUTHENTICATION

Spezifiziert ob Password-Authentifikation erlaubt ist. Betrifft Protokoll 1 & 2

PORT

Gibt den Port an, an den sich der sshd binden soll. Standard ist 22, mehrere Ports koennen uebergeben werden.

PROTOCOL

Gibt die Protokollversionen an, Standard ist 1. 1,2 erlaubt 1 und 2

PROXYCOMMAND

Definiert einen Befehl der benutzt wird um zum Server zu verbinden. Die gesamte Zeile wird als Befehl an /bin/sh uebergeben. %h und %p werden als entfernter Host und Port interpretiert. CheckHostIP ist nicht verfuegbar wenn mit ProxyCommand gearbeitet wird.

REMOTEFORWARD

Forwarded einen TCP/IP Port der entfernten Maschine ueber den verschluesselten Kanal zum angegebenen Host:Port der lokalen Maschine, mehrere Ports koennen angegeben werden, Format ist entfernterPort:Host:lokalerPort

RHOSTSAUTHENTICATION

Gibt an ob rhosts basierte Authentifikation benutzt werden soll. Betrifft nur den Client, wenn der Server kein rhost zulaesst wird dies nicht benutzt.

RHOSTSRSAAUTHENTICATION

Gibt an ob RSA-rhosts basierte Authentifikation benutzt werden soll. Betrifft nur den Client, wenn der Server kein rhost zulaesst wird dies nicht benutzt.

RSAAUTHENTICATION

Gibt an ob RSA authentication benutzt werden soll. Wird nur benutzt wenn der Identityfile dazu existiert und betrifft nur Protokoll 1.

CHALLENGERESPONSEAUTHENTICATION

Gibt an ob Response Authentication benutzt werden soll. Zur Zeit gibt es nur Unterstuetzung fuer skel(1) Authentifizierung

STRICTHOSTKEYCHECKING

Wenn aktiviert, fuegt ssh keine bekannten Schlüssels zu \$HOME/.ssh/known_hosts und \$HOME/.ssh/known_hosts2 und verweigert die Verbindung zu Hosts deren Schlüssel geändert wurden. Wenn auf "ask"

gesetzt muss der Benutzer neue Schluessel bestaetigen, bevor sie hinzugefuegt werden.

USEPRIVILEGEDPORT

Gibt an ob ein privilegierter Port fuer ausgehende Verbindungen genutzt werden soll. Wenn deaktiviert koennen einige aeltere Server mit RhostsAuthentication und RhostsRSAAuthentication nicht mehr kontaktiert werden.

USER

Definiert den Benutzernamen als den man sich einloggen will. Sehr nuetzlich wenn man mehrere Maschinen mit verschiedenen Logins hat.

USERKNOWNHOSTSFILE

Definiert die Datei die anstelle von \$HOME/.ssh/known_hosts verwendet wird

USERSSH

Gibt an das fuer diesen Host rsh/rlogin verwendet werden soll.

XAUTHLOCATION

Definiert den Ort des xauth(1) Programms. Standard ist /usr/X11R6/bin/xauth.

5.3.3 *Using OpenSSH*

5.3.3.1 *Encrypted Tunnels*

OpenSSH can be used to establish an encrypted tunnel to a target through a gateway, running a sshd. This is useful if you want to restrict the access to a target (eg. a PostgreSQL behind a gateway can only be accessed if you have a valid login on the gateway) or if you want to encrypt the tunnel between your host and the gateway. However, the gateway does not have to be a dedicated server, but can also be the target itself. Example 5.3.3.1 shows how to establish a tunnel through a dedicated gateway and the target itself.

```
$ ssh -2 -c aes128-cbc -C stefan@192.168.2.1  
-L 50000:192.168.2.5:5432  
  
$ ssh -2 -c aes128-cbc -C stefan@192.168.2.5  
-L 50000:127.0.0.1:5432  
  
$ psql -U stefan -h 127.0.0.1 -p 50000 MyDatabase
```

Example 38: forwarding PostgreSQL ports with OpenSSH

5.3.3.2 Remote Commands

ssh can also be used as an substitute to rsh, executing commands on a remote computer. This is useful especially for backup/restore purposes.

```
$ ssh stefan@192.168.1.1 "uname -mrs"  
NetBSD 2.0.2 sparc64  
# ssh stefan@192.168.1.1 "dump -0au -f dump0 /home"
```

Example 39: executing commands on a remote computer with ssh

Both commands are executed as user “stefan” on 192.168.1.1, so the first shows the OS name of 192.168.1.1, and not of the local computer. The second command uses dump(8) to make a backup of the local /home/ to the file ~stefan/dump0 on 192.168.1.1.

5.3.4 Hardening OpenSSH

A running sshd can of course be exploited to gain root level access to the system. Security risks can be minimized by following some rules.

Disable password authentication.

A password can be guessed, uncovered by social engineering techniques or cracked in a brute force attack. Disabled passwords

5.4 ENCRYPTED IP-TUNNEL WITH STUNNEL

5.5 ENCRYPTED NETWORK CONNECTIONS WITH VPN

5.6 FIGHTING PORTSCANS WITH PORTSENTRY

Portsentry is a program to monitor connections to several ports of a network interface. If it recognizes a portscan, it can block the source of the scan by changing the route to it to a nonexisting gateway. It can be configured via `/usr/pkg/etc/portsentry.ignore` and `/usr/pkg/etc/portsentry.conf`. The first file lists a number of IP-addresses or subnets that shall be ignored by portsentry. The latter is more interesting and complex, because it is used to configure the portsentry-daemon by the following options:

TCP_PORTS, UDP_PORTS

comma-separated list of ports portsentry shall bind to

ADVANCED_PORTS_TCP, ADVANCED_PORTS_UDP

any unused port below this one will be monitored, usually 1024

ADVANCED_EXCLUDE_TCP, ADVANCED_EXCLUDE_UDP

exclude these ports from monitoring. Useful eg. for Samba, NFS or SMTP.

RESOLVE_HOST

1 enables DNS-lookup, 0 disables it

BLOCK_UDP, BLOCK_TCP

0 = do not block scan, 1 = block scan, 2 = run external command

KILL_ROUTE

command to drop the route to an attacker, `/sbin/route add \ $TARGET \ $ 127.0.0.1` works

SCAN_TRIGGER

number of allowed connections before alarm is given, 0 blocks immediately

portsentry also offers the ability to call an external program on alarm, so you can adjust your packet filter or send an email.

The portsentry configuration should be carefully tested. It is a stupid idea to set SCAN_TRIGGER to 0 and monitor port 443 on a webserver, as a customer looking for a

SSL-version of the site will be blocked immediately.

5.7 MONITORING NETWORK HOSTS WITH ARPWATCH

arpwatch monitors ethernet and IP address pairs and reports activity either to syslogd and / or via email.

arpwatch options:

- D enables debugging and prevents arpwatc from forking into damon mode, no emails are sent either
- F file of the addressdatabase
- I sets the interface
- N specify additional networks to avoid "bogon" messages
- N disable reporting of "bogons"
- R replay network traffic from a file (saved with tcpdump/pcapture)

arpwatch messages to the konsole:

NEW ACTIVITY New address (younger than six month) found

NEW STATION New address found

FLIP FLOP The ethernet address has changed from the most recently seen address to the second most recently seen address.

CHANGED ETHERNET ADDRESS new ethernet address for the host

arpwatch messages to syslog:

ETHERNET BROADCAST

The mac ethernet address of the host is a broadcast address

IP BROADCAST

The ip address of the host is a broadcast address.

BOGON

The source ip address is not local to the local subnet

ETHERNET BROADCAST

source did a broadcast

ETHERNET MISMATCH

The source mac ethernet address didn't match the address inside the arp packet

REUSED OLD ETHERNET ADDRESS

The ethernet address has changed from the most recently seen address to the third (or greater) least recently seen address.

SUPPRESSED DECNET FLIP FLOP

"flip flop" suppressed, one address was DECnet

```
From: arpwatch (Arpwatch)
To: root
Subject: changed ethernet address

        hostname: <unknown>
        ip address: 192.168.2.1
        ethernet address: 0:4:75:ae:50:53
        ethernet vendor: 3 Com Corporation
old ethernet address: 0:a:5e:5:65:eb
old ethernet vendor: <unknown>
        timestamp: Tuesday, September 6, 2005
                    20:29:46 +0200
previous timestamp: Tuesday, September 6, 2005
                    20:27:41 +0200
        delta: 2 minutes
```

```
From: arpwatch (Arpwatch)
To: root
Subject: flip flop

        hostname: <unknown>
        ip address: 192.168.2.1
        ethernet address: 0:a:5e:5:65:eb
        ethernet vendor: <unknown>
old ethernet address: 0:4:75:ae:50:53
old ethernet vendor: 3 Com Corporation
        timestamp: Tuesday, September 6, 2005
                    20:29:46 +0200
previous timestamp: Tuesday, September 6, 2005
                    20:29:46 +0200
        delta: 0 seconds
Example 40: example arpwatch report
```

5.8 FETCHING MAIL ENCRYPTED WITH FETCHMAIL

[pp11]

fetchmail is a program to fetch mail from POP or IMAP servers. POP3 and IMAP can be tunneled over SSL, if the server supports it.

[pp12]

fetchmail can make use of SSL to authenticate the server and prevent password sniffing. fetchmail can be configured either via command line options or via a configuration file (~/.fetchmailrc).

Below are the SSL relevant options. They can either be passed to fetchmail as

command line option or as a keyword in ~/.fetchmailrc.

- SSL / SSL
enables SSL, the server must support SSL
- SSLCERT <NAME> / SSLCERT
name of the client's SSL certificate, some server require it
- SSLKEY <NAME> / SSLKEY
name of the client's private SSL key, some server require it
- SSLPROT <NAME> / SSLPROTO
sets the desired SSL protocol, values are: ssl2, ssl3 and tls1
- SSLCERTCK / SSLCERTCK
strict certificate checking, requires the server certificate to be signed by a trusted local certificate (to -sslcertpath)
- SSLCERTPATH <DIRECTORY> / SSLCERTPATH
directory of local certificates (→ -sslcertck)
- SSLFINGERPRINT / SSLFINGERPRINT
fingerprint of the server, as defined by OpenSSL

An example ~/.fetchmailrc:

```
poll pop3.server.local with proto POP3 user 'lists'  
there with password 'XXX' is 'stefan' here ssl
```

Example 41: example .fetchmailrc

SEE ALSO:

<http://www.catb.org/~esr/fetchmail/>

“BEWARE OF BUGS IN THE ABOVE CODE;
I HAVE ONLY PROVED IT CORRECT,
NOT TRIED IT.”
Donald E. Knuth

6.1 THE CRYPTOGRAPHIC DISK DRIVER CGD

cgd is a pseudo device driver that encrypts blocks on the way to and from disk. This means, that a normal 4.2BSD partition actually does not contain a normal FFS filesystem, since the file system blocks are encrypted. If you want to access the data on the encrypted partition, you have to configure the CGD device. Configuring means, you have to enter the CGD passphrase and mount the now available pseudodevice. Once it is mounted, it can be handled like any other filesystem.

It is the perfect tool to protect data against an intruder who has physical access to your machine, eg. if he “found” your laptop. It is also useful if you are using removable media like USB-sticks or ZIP-Diskettes.

As of 2.0, cgd is not included in the GENERIC-Kernel, so you have to recompile an own kernel with cgd enabled. How to

```
pseudo-device  cgd  4  # cryptographic disk driver
```

Example 42: Kernel config for cgd

Defines 4 cgd devices in the Kernel. Additionally, the device nodes for cgd have to be configured with:

```
$ cd /dev/
$ ./MAKEDEV cgd
```

Example 43: creating cgd devices

cgd supports three ciphers:

- 3des-cbc
- aes-cbc
- blowfish-cbc

Triple DES (3des-cbc) is very slow and is based on the insecure DES cipher, so it should be avoided, AES and Blowfish are faster and newer. However, I prefer AES for my /home/ and Blowfish (due to its speed) for the swap and temp partitions.

The next table shows the `time(1)` values for copying (`cp(1)`) 210 megabyte sized file from /usr/ to a cgd, cfs, mfs (ramdisk) and plain vanilla ffs slice:

cipher/filesystem	real	system
cgd blowfish	9.385	8.346
cgd aes	17.513	13.657
cfs blowfish	82.080	9.203
memory filesystem	3.46	2.15
plain ffs	6.128	2.127

6.1.1 *cgd for a data partition*

Any partition, except /¹, can be encrypted. This chapter will explain how the home partition can be encrypted.

First you have to have a dedicated partition to be encrypted. If the partition already is in use, the data has to be saved either to another partition/disk or backup media. The easiest way is to copy everything beneath /home/ to another filesystem or use `dump(8)` (see chapter ??) to dump the partition to a streamer or CD.

The next step is to wipe the current content of the partition. In chapter 6.5 is described why and how this should be done. You can use `rm -rfP /home/` or a

¹ Somehow the system has to be started.

wiping tool to delete and overwrite the data. This is necessary to prevent the data from being recovered.

Now the dedicated partition is empty and can be configured for cgd. First you have to generate a configuration with cgdconfig:

```
# cgdconfig -g -V ffs -o /etc/cgd/wd0h aes-cbc 256
```

Example 44: configuring cgd for a data partition

This will generate /etc/cgd/wd0h using the AES cipher with 256 bit keylength and a scan for a filesystem as verification method².

6.1.2 *cgd for the swap partition*

The swap partition (which can also be a file), is used to swap out memory from the RAM. It can compromise the security of a system by swapping out sensitive data (passwords, crypto keys) to the hard drive.

To protect the swap, it should be encrypted with cgd.

NetBSD 2.0.2 uses randomkey to generate a random key from /dev/random at startup. This might hang the system if no adequate seed can be computed. This can be worked around by generating some activity using find as described below. NetBSD 2.1 and 3 support urandomkey as option, where /dev/urandom is used, avoiding the hang at startup.

2.0

By using randomkey cgd generates a new key for the swap at system startup, which is lost at shutdown. But that does not matter, since the swap data is not recovered at the next boot.

First you have to create a configuration for the swap partition while the swap partition is not in use:

² cgdconfig will simply check if a FFS is available to see if a valid passphrase has been entered

```
# swapctl -d /dev/wd0b
# cgdconfig -g -k randomkey -o /etc/cgd/wd0b \
                                blowfish-cbc 192

# cat /etc/cgd/wd0b
algorithm blowfish-cbc;
iv-method encblkno;
keylength 192;
verify_method none;
keygen randomkey;
```

Example 45: creating a cgd config for swap

The second step is to save a valid disklabel. The disklabel of the swap partition is lost at shutdown and has to be restored after the partition has been configured.

This can be achieved by:

1. configure the cgd device
2. create swap disklabel
3. save the disklabel
4. load the saved disklabel
5. configure the swap device

```

1 # cgdconfig cgd1 /dev/wd0b
2 # disklabel -i /dev/cgd1a
partition> a
Filesystem type [?] [swap]: swap
Start offset ('x' to start after partition 'x')
                                [0c, 0s, 0M]:
Partition size ('$' for all remaining)
                                [755.508c, 1547280s, 755.508M]:
partition> W
Label disk [n]? y
Label written
partition> Q
3 # disklabel -r /dev/cgd1a > /etc/cgd/swap.disklabel
4 # disklabel -R -r cgd1 /etc/cgd/swap.disklabel
5 # swapctl -a /dev/cgd1a
6 # swapctl -l
Device      512-blocks Used Avail Capacity Priority
/dev/cgd1a  1547280    0 1547280    0%  0

```

Example 46: creating a cgd disklabel for swap

This procedure must now be automated in `/etc/rc.local` to be run at every startup:

```

/usr/bin/find /etc/ | /usr/bin/xargs /usr/bin/md4 \
                    > /dev/null

/sbin/cgdconfig cgd1 /dev/wd0b
/sbin/disklabel -R -r cgd1 /etc/cgd/swap.disklabel
/sbin/swapctl -a /dev/cgd1a

```

Example 47: configuring cgd for swap via `/etc/rc.local`

The first line runs a `find(1)` through `/etc/` and pipes each file to `md4`, which calculates a MD4 checksum. This is simply used to generate a random seed, so the key generation will not hang and can be avoided if `urandomkey` is used.

Finally, the cgd-device `cgd1` is configured, the disklabel written to the cgd-partition and configured as swapspace with `swapctl`.

6.1.3 *cgd for the tmp partition*

Temporary files should also be protected by using a memory file system (mfs, also called ramdisk) or by encrypting /tmp/ with cgd.

To use cgd for /tmp/ you have to generate a configuration file that uses randomkey as keygen and none as verify_method. I suggest using Blowfish as cipher, since it is the fastest available.

```
# cgdconfig -g -k randomkey -o /etc/cgd/wd0h \
                                blowfish-cbc 192
# cat /etc/cgd/wd0h
algorithm blowfish-cbc;
iv-method encblkno;
keylength 192;
verify_method none;
keygen randomkey;
#
```

Example 48: creating a cgd config for /tmp/

To configure /tmp/ at startup, one can use /etc/rc.local as follows:

```
/sbin/cgdconfig cgd2 /dev/wd0h && /sbin/newfs /dev/cgd2a
/sbin/mount /dev/cgd2a /tmp
/bin/chmod 777 /tmp/
```

Example 49: configuring cgd for /tmp/ via /etc/rc.local

The first line configures the cgd-device and creates a new filesystem on it. A /tmp/ partition usually is very small (< 300MB) so this will not take too long. The second line mounts /dev/cgd2a as /tmp, and finally the right permissions are set to /tmp/.

6.1.4 *cgd for an ISO 9660 (CDROM) Image*

NetBSD uses so called virtual node drivers to provide a disk-like interface to a file. This can for example be used to mount an image of a CD or a HD.

So cgd can be used, in conjunction with the virtual node driver, to encrypt a file as image. That image can either be burned to a CD or used as an encrypted container eg. on a Samba or NFS exported directory or on removable media like USB sticks.

```
1 $ dd if=/dev/zero of=image bs=2m count=90
2 # vnconfig -c vnd0 image
3 # cgdconfig -g -o image.cgd blowfish-cbc 128
4 # cgdconfig -V re-enter cgd3 /dev/vnd0a image.cgd
5 # newfs /dev/cgd3a
6 # mount /dev/cgd3a /mnt/
  ...
7 # umount /dev/cgd3a
8 # cgdconfig -u cgd3
9 # vnconfig -u vnd0
```

Example 50: creating a cgd encrypted file image

1. reads /dev/zero to generate a file called image, a blocksize of 2 megabytes is used and 90 blocks are generated, so the file be $90 * 2m = 180m$ big.
2. vnconfig configures image as /dev/vnd0, so it can be accessed like a block device
3. generates the configuration file for cgd with blowfish as cipher and a keylength of 128, writes it to image.cgd
4. configure the image (/dev/vnd0a as cgd3, since this is the first time, “re-enter” is used as verification, cgdconfig asks for the passphrase to be used)
5. once the cgd-device is configured, a new filesystem can be constructed
6. the image is ready to be mounted and can be used as desired
7. unmount the image
8. destroy the cgd-device
9. destroy the virtual node, image and image.cgd can now be configured

SEE ALSO:

cgd cgdconfig vnd vndconfig [Chapter 14. The Cryptographic Device Driver (CGD)]guide
[Part V. Building the System]guide

cgdswap
cgd

6.2 THE CRYPTOGRAPHIC FILESYSTEM CFS

6.2.1 *What is cfs?*

[pp21]

Cfs is the so called cryptographic filesystem which implements encryption at system level through a standard file system interface to encrypted files. Files in these directories (as well as their pathname components) are transparently encrypted and decrypted with the specified key without further user intervention; cleartext is never stored on a disk or sent to a remote file server. It is implemented entirely at user level, as a local NFS server running on the client machine's loopback interface.

6.2.2 *Installing and Configuring cfs*

cfs is available via pkgsrc as [pp21].

After installation finished you have to create, configure and export a working directory:

```
$ mkdir /null
$ chmod 0 /null
$ echo /null localhost >> /etc/exports
```

Example 51: creating a cfs workdir

Now [re]start mountd and cfsd, configure a cfs mount point and mount it:

```
$ /etc/rc.d/mountd [re]start && /usr/pkg/sbin/cfsd
$ mkdir /crypt
$ mount -o intr,-2,-w=4096,-r=4096 127.0.0.1:/null /crypt
```

Example 52: mounting cfs workdir

finally mountd and cfsd should be started via /etc/rc.conf resp. /etc/rc.local, also you can add the mount command to the latter. Now it is time to create the encrypted

directory, the `-b` option specifies "Blowfish" algorithm. Please refer to man-page for other algorithms, such as 3DES or MacGuffin.

```
$ cfs\_mkdir -b /encrypted
```

Example 53: creating a encrypted directory for cfs

`cfs_mkdir` now asks for an at least 16 character long passphrase which should be of course very strong, e.g. use numbers, capitalization and special characters.

`cfs` is now configured and ready for use. To mount your encrypted directory supply

```
$ cfs_attach /encrypted usable_cfsdir
```

Example 54: mounting the encrypted directory for cfs

and enter your passphrase. It is now attached as `/encrypted/usable_cfs` and can be used like a regular directory, e.g. you can copy files into it.

For example, if you copy `/etc/motd` to `/encrypted/usable\_cfs` you can access `/encrypted/usable\_cfs/motd` as usual, because this is the decrypted interface to the "real" encrypted file, now residing in `/crypt/`

To detach the encrypted directory use

```
$ cfs_detach usable_cfsdir
```

Example 55: detach the encrypted cfs directory

Now you cannot access `/encrypted/usable\_cfs/` anymore

6.2.3 Using cfs on Removable Media

It is possible to use a removable media with `cfs`, so for example you can burn your private fotos onto a CDROM in an encrypted way and simply mount and `cfs_attach` it to access the files. I'll shortly describe how to burn a `cfs`-directory to a CD:

- create a new `cfs`-directory for the pictures
- attach it and copy the images into it

- detach it and create an ISO Image using `cdrecord`
- burn that image to cd

```
$ cfs\_mkdir -b /fotos-dec03
$ cfs\_attach /fotos-dec03 fotos
$ cfs\_detach fotos
$ mkisofs -o fotos.iso -rTJ /fotos-dec03
```

Example 56: using `cfs` on a CD

Using that CD is very simple, mount it to e.g. `/mnt` and "`cfs_attach /mnt/fotos-dec03 fotos`" to access the images.

6.2.4 Backups and `cfs`

One problem regarding backups is that `cfs` also encrypts file- and directorynames, so it is hard to identify several files. One possible solution is to backup the files unencrypted in an attached `cfs`-directory. But then you should also encrypt your backups, for example with `mrcrypt` or `GnuPG`, which are both included in `pkgsrc`. In case you have to restore your files, this must also be done into the attached encrypted directory. But it is also possible to backup and restore the still encrypted files, residing in `/crypt`. This can be done by backing up the whole directory or via incremental backups using `dump(8)` and its `dumplevels` or `find(1)` and its date options and other backup software, such as `pax(1)` or `cpio(1)`.

6.2.5 `cgd` versus `cfs`

So which one is better, `cgd` or `cfs`? As usually, that question can only be answered with a "depends on". First you have to clear out your requirements and then you can check which program fits your needs better.

If you have single user machine that should be protected in case it is lost or stolen (eg. a laptop) `cgd` should be sufficient, especially since it is faster than `cfs`.

But once a `cgd` partition is mounted, it behaves like a plain NetBSD file system, so there is no protection against other users. If you are interested in encrypting your `/home/` but also want to protect sensitive data (mailbox, PGP keys, some data directories) from intruders in an untrustworthy network, use both.

On my personal laptop, `/home/`, `/tmp/` and `swap` are encrypted with `cgd`. Additionally, my `~/mail`, `~/gnupg`, `~/silc` and some data directories are encrypted with `cfs`. If I have to use that Laptop eg. on a talk I can mount my `cgd` encrypted home, but my mailbox, GnuPG and other data is still protected by the not mounted `cfs`.

6.3 SYMMETRIC ENCRYPTION WITH MCRYPT

`mdecrypt` is a program for simple symmetric encryption of files. One can pass several options determining the way of encryption, eg:

- G generate OpenPGP packets
- D decrypt file
- A *algo* use encryption algorithm *ALGO*
- s *size* size of the key
- M *mode* encryptionmode
- Z (un)compress with GZIP
- P (un)compress with BZIP2
- B baremode, don't save metadata like mode or algorithm
- FLUSH flush data immediately, useful if piped

```
$ mdecrypt dmesg.txt
```

Example 57: encrypting a file with `mdecrypt` (standard)

will ask for your passphrase and encrypt `dmesg.txt` with the standard options `rijndael-128` (AES), `SHA1` checksum, CBC mode. If you want to encrypt files consisting of natural texts, it is a good idea to compress it before encryption like that:

```
$ mdecrypt -a rijndael-256 -h sha512 -p foo.bar
```

Example 58: encrypting a file with `mdecrypt`

6.4 THE GNU PRIVACY GUARD GPG

6.5 DELETING DATA THE SECURE WAY (“WIPING”)

This subsection is about magnetical devices, it doesn't apply to eg. flash-RAM or CD/DVDs.

A usual `rm` doesn't delete a file really, but dereferences his entry in the filesystem, so the use blocks are marked empty and can be overwritten. Until the file actually is overwritten, it can be recovered for example with `fsdb(8)` or more advanced data recovery tools.

If you encrypt a file eg with `mcrypt` (see Chapter 6.3, “Symmetric Encryption with `mcrypt`” on page 99) you have to remove the remaining unencrypted one in a secure manner. Electrophysical there are only two ways of achieving that, deploying the device to Curie-temperature³ or a strong⁴ rotating magnetic field⁵, that will swirl the magnetic structures of ferromagnetic matter. Both ways have problems, they are expensive (unless you own a personal blast furnace or magnetic resonance tomograph) and they destroy the magnetic device.

There is another way to secure erase data on magnetic devices, the data just has to be overwritten several times by random generated data to make it unreadable. There exist several standards how this has to be done, eg. Peter Gutman or US-DoD sanitizing standard 5220.22-M.

NetBSD's `rm` offers the option `-P` to overwrite the files to be deleted with `0x00`, `0xFF` and random data, which isn't really secure, it only provides protection against `fsdb(8)`

Another way might be the use of `dd` to fill a filesystem or wipe a rawdevice, in case you want to wipe a whole harddrive:

```
$ dd if=/dev/urandom of=/dev/wd1d bs=2m
```

Example 59: wiping a raw device

will get pseudorandom⁶ written directly to the second harddrive. It is required to use more than a single pass, since it is possible to read the former imprinting of

³ the temperature where ferromagnetic matter loses magnetism, this are about 1000K

⁴ 1.5 Tesla

⁵ sold as 'Degausser'

⁶ you don't need real cryptographic randomness for wiping, it is sufficient to have non-linear data

magnetic matter and each pass of random data will make this harder. This also the reason to use non-linear data. So one can use a simple loop to wipe a harddrive:

```
$ for i in `seq 1 5`  
do  
dd if=/dev/urandom of=/dev/wd1d bs=2m  
dd if=/dev/zero of=/dev/wd1d bs=2m  
done
```

Example 60: wiping a raw device in a loop

this will wipe the drive five times with randomdata and zeros (zeros are of course faster to generate) and should be enough to make the drive unreadable by most peoples and agencies.

Additionally, there is `pkgsrc/sysutils/wipe`, a wiper that implements the US DoD secure deletion standard and is therefor able to wipe files.

```
$ wipe -r -k -l2 -x -p5 /home/
```

Example 61: wiping files with wipe

will wipe `/home/` recursive, locks the file that is being wiped, pseudo data will be seeded once per pass with five loops of pseudodata.

6.5.1 *Disadvantages of Wiping*

Of course wiping will prevent recovering of files, so you should be sure to wipe only files you want to delete.

Wiping a file requires the system to generate random data at least as large as the files to be deleted, so the wiper will generate a high load und take a long time⁷

On a Athlon 800, 1GB Ram, ATA100 HD:

⁷ several days to US-DoD-wipe a 20GB harddrive on a Pentium III-500 with ATA66-HDD

```
$ dd if=/dev/zero of=wipe.me bs=1m count=10
$ time wipe -r -k -l2 -x -p5 wipe.me
    33.7 real 0.44 user 26.78 sys
$
```

Example 62: benchmarking wipe

generates a load of 6.

To wipe a drive it must be writable, so a defect drive (eg a headcrashed one) cannot be wiped and the file system must not be log-structured or block-journaling, since those FS reallocate sectors on write. Even on a traditional FFS it cannot be guaranteed that the wiper data really is written to the drive.

If you are sure that you need to wipe your data, it is a good idea to encrypt it with `cgd` (see Chapter [cgd](#), “[Bibliography](#)” on page [129](#)).

Part III

THIRD PARTY SOFTWARE

THIRD PARTY SOFTWARE

“ COMPUTER SIND WIE ALLE INSTRUMENTE
NICH T WERTFREI, SONDERN ERBEN IHRE
WERTE VON DER GESELLSCHAFT,
IN DER SIE EINGEBETTET SIND.”
Joseph Weizenbaum

7.1 THE NETWORK INTRUSION DETECTION SYSTEM SNORT

7.2 THE APACHE WEB SERVER

7.3 THE POSTGRESQL DATABASE SYSTEM

7.4 THE X WINDOWS SYSTEM

7.4.1 *Locking the Screen*

To lock the X screen, a third party package is required, be it

XScreensaver [\[pp22\]](#) Xlockmore [\[pp24\]](#) or Xlockmore-light [\[pp23\]](#).

[\[pp25\]](#). [\[pp26\]](#)

All three offer some kind of screensaver that can lock the screen. XScreensaver also has XScreensaver-demo [\[pp26\]](#) which can be used to configure it's options like blank and lockout timer.

[\[pp22\]](#)

[\[pp24\]](#)

[\[pp23\]](#)

Additionally, there is xautolock [pp25], which can start any given program after a timeout of inactivity has been triggered. Xscreensaver can also be started from a terminal

```
$ xscreensaver-command -lock
```

Example 63: starting XScreensaver to lock the screen

or some windows manager offer a shortcut to lock the screen, like XFCE does with [CTRL]+[ALT]+[DEL].

By default, a X session can be killed with [CTRL]+[ALT]+[BSP], so an intruder could simply kill X to get access to your konsole, where X has been started. This can be disabled with Option "dontzap" in /etc/X11/XF86Config or by starting X that way:
\$ xinit ; logout to logout from the terminal once X quits.

X should not be installed if it is not needed, eg. on a WWW server. NetBSD allows you to select which sets should be installed when installing it via sysinst

XFree uses the X font server xfs to supply fonts to X Window System display server. It is usually started via /etc/rc.conf and listens as daemon on port 7100. If it is running, access should be restricted with ipf.

7.5 WEB BROWSER AND SECURITY

A webbrowser can be a very dangerous program if it is integrated to deep into a system and offers to much programming abilities. Some browser include a lot of plugins and languages that can be exploited by websites. Here also applies the idea of a lean system, so a simple browser without programming languages is more secure than one with.

The security threads of web browsers:

running programs / scripts

Some browsers support scripting or programming languages like JavaScript or Java and can run programs. These program can of course be malware.

disable scripting languages

cookies

Cookies are used to store data in a persistent way, the data is stored on the client within a small text file (called cookie) or on server side and the cookie is used to

identify the user across page requests. This is useful where a user id is needed (eg. webshop or webmail) but can be misused to track a user and spy on him.

disable or restrict cookies or at least, delete them if no longer needed

plugins

Plugins are programs installed to enhance the browser. They can be used to start RealPlayer, mplayer or play flash movies. Since they are a program, they can be malware too. They might also be trojaned.

disable or do not install plugins, use another environment for the browser

faked URLs

URLs can include a username, like `http://foo@bar.com`, this is sometimes misused by phisher who want to feign another address like that:

`http://login.internetmarketplace.foo#blahbla@phishermans.server.`

Use your own bookmarks or enter the URL manually, don't trust eMails or other sites

As afore said, a browser can be subject to trojaned programs. These trojans will have the same permissions as your user, so it is a good strategy to use another account exclusively for the browser. If that account cannot access your data, the browser can't do so, too¹.

7.5.1 Opera

Opera is a very lean web browser that also includes a mail, news and irc client. It is available via pkgsrc as a Linux binary package and offers a lot of functions to enhance security and privacy. Additionally it was the first browser to use a multiple document interface (also called tabbed browsing) where one instance of Opera uses multiple windows. It is available for several hardware and software platforms.

Opera offers a "Quick Preferences" Menu under "Tools" or the shortcut F12. This Menu can be used to block popups, Dis-/Enable Java, JavaScript and plugins, cookies, referer² logging and proxy server.

To enable maximum security, disable programming languages like Java and JavaScript. Referer logging and cookies should be disabled too, but this can influence sites where you have to login, eg. webshops.

Since I begun to use Opera in 1999, it offers something very special: the "delete private data" option to clear some data. It is nowadays available as option of the "Tools" menu and allows one to use Cookies, the cache, visited addresses, bookmarks

¹ unless a root-exploit happens

² someone misspelled referrer in a RFC, so it got stuck

time and so on.

Opera now offers “wand” to manage passwords for several logins, however, in my opinion wand should be disabled.

7.5.2 *Firefox*

7.5.3 *Netscape/Mozilla*

7.5.4 *Textbrowsers: lynx / links / w3m*

7.5.5 *Other Browsers*

chimera
dillo
epiphany gecko galeon
konqueror
kazehakase
About Kazehakase

Kazehakase is a browser with gecko engine like Epiphany or Galeon. Main features
Tabbed browsing Remote bookmark (e.g. RSS) in menu or sidebar It is able to update
periodically In Japan, there are other remote bookmarks, for instance, LIRS (In
Japanese), hina-di (In Japanese) , Kazehakase can handle these ones. Variable UI
(menus, toolbar etc.) on user level Customizable mouse gestures Customizable key
accelerator grail
amaya

7.5.6 *WWW Proxies*

A proxy (latin *proximus*: nearest, closest) is a program that is used to route network
connections through it. It is often transparent and mainly used to reduce network
load by offering a central cache. A WWW proxy can often also be used to enhance
security and/or privacy by filtering banners, ads, cookies and different headers.

7.5.6.1 *The Internet Junkbuster & Privoxy*

7.5.6.2 *WWWOFFLE*

7.5.6.3 *tinyproxy*

7.5.6.4 *SwiftSurf*

7.5.6.5 *SQUID*

Part iv

KNOW YOUR ENEMY

SI VIS PACEM, PARA BELLUM.
Vegetius

8.1 USING VULNERABILITY SCANNER

8.1.1 *Nessus*

8.1.2 *Nikto*

8.1.3 *Hackbot*

8.1.4 *Amap*

KNOW YOUR ENEMY: INFORMATION ABOUT SECURITY HOLES

“PLAUDITE, AMICI, COMOEDIA FINITA EST!”
LUDWIG VAN BEETHOVEN

9.1 NETBSD AND SECURITY

NetBSD security issues are handled by the NetBSD security officer¹ or the NetBSD security alert team². If you want to report a security problem, you can use `send-pr` (1) or email to the security alert team, where you can use PGP Key `oxF8376205`.

Security advisories regarding the base system are sent to the ‘netbsd-announce’-mailing list and additionally archived on the security website.

Security issues for `pkgsrc` packages should be resolved via `audit-packages` (see Chapter 3.14, “[Check for Insecure Packages with audit-packages](#)” on page 54)

A list of other sources of information is available in Appendix B

SEE ALSO:

Appendix B, “[Security Relevant Sources of Information](#)” on page 123
[sec](#)

<http://www.netbsd.org/MailingLists/#netbsd-announce>

[Check for Insecure Packages with audit-packages](#), p. 54

¹ <security-officer@NetBSD.org>, PGP: `oxF8376205`

² <security-alert@NetBSD.org>

“NICHTS IST GETAN, WENN NOCH ETWAS ZU TUN ÜBRIG IST.”
Carl Friedrich Gauß

10.1 HARDWARE INDEPENDENT

10.1.1 Enclosures

There are special enclosures available, that are conform to either industrial and/or security standards, like [DIN60529](#) defined IP-classes. It might be useful to protect a computer with an industrial enclosure, if it is exposed to public access and/or industrial hazards, like electricity, dust or chemical material.

Some standard enclosures and almost all modern laptops offer a so called “Kensington”-lock slot, where standardized locks can be attached to.

Some standard enclosures also offer special locks applicable to standard screws and other ways to secure them.

10.1.2 Removable Disks

10.1.3 Redundancy

Part v

APPENDICES AND LISTS



CODE & EXAMPLES

"IF YOU SPEND MORE ON COFFEE
THAN ON IT SECURITY, THEN YOU
WILL BE HACKED. WHAT'S MORE,
YOU DESERVE TO BE HACKED."
Richard Clarke

check versions of installed packages	30
password generation with apg	34
generate passwords with OpenSSL	35
password hashes with pwhash	35
benchmarking pwhash	36
/etc/passwd.conf	37
locking the terminal with lock	38
running a process in the background with & and nohup	39
crontab-entries for maintenance scripts	40
process status with ps	43
network status with netstat	45
setting the date with ntpdate	46
configuring ntpdate via rc.conf	46
crontab	47
anacron crontab	48
scheduling programs with at	49
setting file ownership and permissions	51
checking the kern.securelevel	51
logging to syslogd with logger	54
installing audit-packages	55
creating a ramdisk and running backups	58
restoring backups with restore	59
configuring the kernel for VERIEXEC	61
creating the veriexec devices	62
generate a filesystem fingerprint with mtree	64
example mtree fingerprint	64
comparing a mtree fingerprint	64
mtree report	65

- aide.conf 66
- AIDE makro 67
- AIDE configuration 67
- initializing AIDE 68
- checking a filesystem with AIDE 68
- excerpt from a AIDE database 69
- AIDE report 69
- running chkrootkit 70
- scanning the net with ddos-scan 71
- forwarding PostgreSQL ports with OpenSSH 82
- executing commands on a remote computer with ssh 82
- example arpswatch report 86
- example .fetchmailrc 87
- Kernel config for cgd 89
- creating cgd devices 90
- configuring cgd for a data partition 91
- creating a cgd config for swap 92
- creating a cgd disklabel for swap 93
- configuring cgd for swap via /etc/rc.local 93
- creating a cgd config for /tmp/ 94
- configuring cgd for /tmp/ via /etc/rc.local 94
- creating a cgd encrypted file image 95
- creating a cfs workdir 96
- mounting cfs workdir 96
- creating a encrypted directory for cfs 97
- mounting the encrypted directory for cfs 97
- detach the encrypted cfs directory 97
- using cfs on a CD 98
- encrypting a file with mrcrypt (standard) 99
- encrypting a file with mrcrypt 99
- wiping a raw device 100
- wiping a raw device in a loop 101
- wiping files with wipe 101
- benchmarking wipe 102
- starting XScreenSaver to lock the screen 106

SECURITY RELEVANT SOURCES OF INFORMATION

“ACH GOTT! DIE KUNST IST LANG!
UND KURZ IST UNSER LEBEN.”
Johann Wolfgang von Goethe

There are several other sources of information including, but not limited to, NetBSD:

- **FreeBSD Security Information**
<http://www.freebsd.org/security/index.html>
- **OpenBSD Security**
<http://openbsd.org/security.html>
- **Computer Emergency Response Team / Coordination Center (CERT/CC)**
publishes Advisories & Incident Notes, Statistics, Tech Tips, Research, Analysis and much more
<http://www.cert.org/>
- **Deutsches Forschungsnetz - Computer Emergency Response Team (DFN-CERT)**
offers CERT services for Germany's National Research and Education Network (Deutsches Forschungsnetz, DFN), in German
<http://www.dfn-cert.de/>
- **Bugtraq**
BugTraq is a full disclosure moderated mailing list for the *detailed* discussion and announcement of computer security vulnerabilities: what they are, how to exploit them, and how to fix them.
FAQ: <http://seclists.org/about/bugtraq.txt>
Archive: <http://seclists.org/lists/bugtraq/>
- **full disclosure**
A mailing list regarding security issues, without moderation.

FAQ: <http://lists.grok.org.uk/full-disclosure-charter.html>

Archive: <http://seclists.org/lists/fulldisclosure/>

- **Seclists.Org Security Mailing List Archive**
lists security related lists and describes them, good portal
<http://seclists.org/>
- **Astalavista**
Several security related topics, like exploits, tutorials and news
<http://astalavista.com/>
- **The Information Warfare Site (IWS)**
IWS is an online resource that aims to stimulate debate on a variety of issues involving information security, information operations, computer network operations, homeland security and more. It is the aim of the site to develop a special emphasis on offensive and defensive information operations.
<http://www.iwar.org.uk/>
- **SANS Newsletters**
Computer Security Newsletters and Digests by SANS
<http://www.sans.org/newsletters/>
- **Packet Storm**
Packet Storm offers an abundant resource of up-to-date and historical security tools, exploits, and advisories.
<http://www.packetstormsecurity.org/>
- **2600**
The Hacker Quarterly
<http://www.2600.org/>
- **Phrack**
Hacker Magazine
<http://www.phrack.org/>
- **Die Datenschleuder**
Chaos Computer Club's Magazine - in German, online archive available
<http://ds.ccc.de/>
- **de.comp.security.misc, de.comp.security.firewall**
Two German newsgroups regarding security and firewall issues
Google Archive: <http://groups.google.de/group/de.comp.security.misc>
Google Archive: <http://groups.google.de/group/de.comp.security.firewall>
FAQ: <http://www.iks-jena.de/mitarb/lutz/usenet/Firewall.html>



PACKAGES AVAILABLE IN PKGSRG

- [pp1] **crack**
The 'Sensible' Unix Password Cracker
pkgsrc/security/crack
<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/security/crack/README.html>
<http://www.crypticide.com/users/alecm/>
- [pp2] **chkrootkit**
Locally checks for signs of a rootkit
pkgsrc/security/chkrootkit
<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/security/chkrootkit/README.html>
<http://www.chkrootkit.org/>
- [pp3] **apg**
Tool set for random password generation
pkgsrc/security/apg
<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/security/apg/README.html>
<http://www.adel.nursat.kz/apg/>
- [pp4] **hydra**
Login password cracker
pkgsrc/security/hydra
<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/security/hydra/README.html>
<http://www.thc.org/thc-hydra/>
- [pp5] **john**
Unix Password Cracker
pkgsrc/security/john
<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/security/john/README.html>

<http://www.openwall.com/john/>

[pp6] **AIDE**

Free intrusion detection system for checking file integrity

pkgsrc/security/aide

<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/security/aide/README.html>

<http://sourceforge.net/projects/aide>

[pp7] **snort**

The Open Source Network Intrusion Detection System

pkgsrc/net/snort

<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/net/snort/README.html>

<http://www.snort.org/>

[pp8] **apache2**

Apache HTTP (Web) server, version 2

pkgsrc/www/apache2

<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/www/apache2/README.html>

<http://httpd.apache.org>

[pp9] **postgresql**

Robust, next generation, object-relational DBMS

pkgsrc/databases/postgresql80

<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/databases/postgresql80/README.html>

<http://www.postgresql.org>

[pp10] **arpwatch**

Monitor ARP activity on a network, maintaining a database

pkgsrc/net/arpwatch

<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/net/arpwatch/README.html>

<http://www-nrg.ee.lbl.gov/>

[pp11] **fetchmail**

Batch mail retrieval/forwarding utility for pop2, pop3, apop, imap

pkgsrc/mail/fetchmail

<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/mail/fetchmail/README.html>

<http://catb.org/esr/fetchmail/>

[pp12] **fetchmailconf**

Configuration tool for fetchmail

pkgsrc/mail/fetchmailconf

<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/mail/fetchmailconf/README.html>

<http://catb.org/esr/fetchmailconf/>

[pp13] **anacron**

Configuration tool for fetchmail

pkgsrc/time/anacron

<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/time/anacron/README.html>
<http://anacron.sourceforge.net/>

- [pp14] **slurm**
 Realtime traffic statistics
 pkgsrc/net/slurm
<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/net/slurm/README.html>
<http://www.raisdorf.net/projects/slurm/>

- [pp15] **bmon**
 Bmon is an interface bandwidth monitor
 pkgsrc/net/bmon
<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/net/bmon/README.html>
<http://people.suug.ch/~tgr/bmon/>

- [pp16] **pkg_chk**
 Check installed package versions against pkgsrc
 pkgsrc/pkgtools/pkg_chk
ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/pkgtools/pkg_chk/README.html

- [pp17] **audit-packages**
 Tools to show vulnerabilities in installed packages
 pkgsrc/security/audit-packages
<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/security/audit-packages/README.html>

- [pp18] **sudo**
 Allow others to run commands as root
 pkgsrc/security/sudo
<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/security/sudo/README.html>
<http://www.courtesan.com/sudo/>

- [pp19] **ddos-scan**
 Scan for a limited set of distributed denial of service (ddos) agents
 pkgsrc/security/ddos-scan
<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/security/ddos-scan/README.html>
<http://staff.washington.edu/dittrich/>

- [pp20] **adjustkernel**
 Generate a NetBSD kernel config
 pkgsrc/sysutils/adjustkernel
<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/sysutils/adjustkernel/README.html>

- [pp21] **cfs**
 Encrypting file system, using NFS as its interface
 pkgsrc/security/cfs
<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/security/cfs/README.html>
<http://www.crypto.com/software/>

- [pp22] **xscreensaver**

Screen saver and locker for the X window system

pkgsrc/x11/xscreensaver

<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/x11/xscreensaver/README.html>

<http://www.jwz.org/xscreensaver/>

[pp23] **xlockmore-lite**

lite version of xlockmore

pkgsrc/x11/xlockmore-lite

<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/x11/xlockmore-lite/README.html>

<http://www.tux.org/~bagleyd/xlockmore.html>

[pp24] **xlockmore**

Like the XLock session-locker/screensaver

pkgsrc/x11/xlockmore

<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/x11/xlockmore/README.html>

<http://www.tux.org/~bagleyd/xlockmore.html>

[pp25] **xautolock**

Activate xlock after a user defined time of inactivity

pkgsrc/x11/xautolock

<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/x11/xautolock/README.html>

[pp26] **xscreensaver-demo**

Demo viewer and configuration tool for xscreensaver

pkgsrc/x11/xscreensaver-demo

<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/x11/xscreensaver-demo/README.html>

<http://www.jwz.org/xscreensaver/>

[pp27] **gpart**

<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/x11/xscreensaver-demo/README.html>

<http://www.jwz.org/xscreensaver/>

[pp28] **wipe**

Secure data destruction

pkgsrc/

<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc//README.html>

[pp29] **destroy**

Program to securely destroy files on the hard disk

pkgsrc/sysutils/destroy

<ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc//README.html>

D

BIBLIOGRAPHY

“NUR WENIGE WISSEN,
WIE VIEL MAN WISSEN MUSS,
UM ZU WISSEN,
WIE WENIG MAN WEISS.”
Werner Heisenberg

cgd

DOWDESWELL, Roland C.
The Cryptographic Disk Driver
<http://www.imrry.org/~elric/cgd/cgd.pdf>

guide

The NetBSD Developers
The NetBSD Guide
<http://netbsd.org/guide/en/>

vepaper

LYMN, Brett & FINK, Jay
Preventing the Unauthorised Binary
<http://www.users.on.net/~blymn/veriexec/sexec.html>

koma

KOHM, Markus und MORAWSKI, Jens-Uwe
Das KOMA-Script Paket
<http://www.tug.org/tex-archive/macros/latex/contrib/koma-script/scrguide.pdf>

FIPS-181

National Institute of Standards and Technology
Standard for Automated Password Generator
<http://www.itl.nist.gov/fipspubs/fip181.htm>

vk

CLAUSEWITZ, Carl von
Vom Kriege
Dümmlers Verlag, Berlin, 1832
<http://www.clausewitz.com/CWZHOME/VomKriege/VKTOC.htm>

phish

GRÖTKER, Ralf
Outsourcing beim Banküberfall
Phishing-Mails: Netzwerkökonomie statt organisierte Kriminalität
<http://www.heise.de/tp/r4/artikel/20/20871/1.html>

cert9907

CERT Coordination Center Incident Note
Distributed Denial of Service Tools
http://www.cert.org/incident_notes/IN-99-07.html

ca200001

CERT Coordination Center & Federal Computer Incident Response Capability
Denial-of-Service Developments
<http://www.cert.org/advisories/CA-2000-01.html>

lkm

Benedikt Meurer
Introduction to NetBSD loadable kernel modules
<http://www.home.unix-ag.org/bmeurer/NetBSD/howto-lkm.html>

phrack-lkm

truff
Phrack Volume 0x0b, Issue 0x3d, Phile #0x0a
Infecting loadable kernel modules
http://www.phrack.org/phrack/61/p61-oxoa_Infecting_Loadable_Kernel_Modules.txt

gutman

Peter Gutmann
Secure Deletion of Data from Magnetic and Solid-State Memory
http://www.cs.auckland.ac.nz/pgut001/pubs/secure_del.html

rad

MÖLLER, Erhart
Fachhochschule Aachen
Protective Measures Against Compromising Electro Magnetic Radiation Emitted by Video Display Terminals
Published in Phrack Magazine #44

<http://www.shmoo.com/tempest/PHRACK44-11>

jarg

RAYMOND, Eric S. (*editor*)
The Jargon File, version 4.4.7
<http://www.catb.org/esr/jargon/>

radjam

Stefan Krempf
secret service bugging attacks: Target: Big Companies
 c't 4/99, page 182 - secret service bugging attacks
<http://www.heise.de/ct/english/99/04/182/>

DIN61508

Deutsches Institut für Normung e. V. (DIN) & Verband der Elektrotechnik
 Elektronik Informationstechnik e.V. (VDE)
*DIN EN 61508 (VDE 0803): Funktionale Sicherheit sicherheitsbezogener
 elektrischer/elektronischer/programmierbarer elektronischer Systeme*

DIN60529

Deutsches Institut für Normung e. V. (DIN)
DIN EN 60529: Schutzarten durch Gehäuse

sef

GRANGER, Sarah
Social Engineering Fundamentals
 Part 1: <http://www.securityfocus.com/infocus/1527>
 Part 2: <http://online.securityfocus.com/infocus/1533>

se1

'Harl'
People Hacking: The Psychology of Social Engineering
[http://cybercrimes.net/Property/Hacking/Social
 Engineering/PsychSocEng/PsySocEng.html](http://cybercrimes.net/Property/Hacking/SocialEngineering/PsychSocEng/PsySocEng.html)

wg

BADHAM, John (Director)
WarGames (1983) (Movie)
<http://imdb.com/title/tt0086567/>

km1

MITNICK, Kevin
The Art of Deception: Controlling the Human Element of Security

km2

MITNICK, Kevin

The Art Of Intrusion: The Real Stories Behind The Exploits Of Hackers, Intruders And Deceivers

uroot

ALTUNERGIL, Oktay

Understanding Rootkits

<http://www.linuxdevcenter.com/pub/a/linux/2001/12/14/rootkit.html>

sroot

ALTUNERGIL, Oktay

Scanning for Rootkits

<http://www.linuxdevcenter.com/pub/a/linux/2002/02/07/rootkits.html>

sec

The NetBSD Foundation

Security and NetBSD

<http://www.netbsd.org/Security/>

ff

WENNAMCHER, Alexandre

File Flags Proposal

<http://www.free-x.ch/pub/proposal.txt>

cfs

SCHUMACHER, Stefan

Encrypted filesystem with cfs

www.net-tex.de/unix/cfs.html

veri

SCHUMACHER, Stefan

verified executables

<http://www.net-tex.de/unix/veriexec.html>

ids

SCHUMACHER, Stefan

Einbruchserkennung in Netzwerke mit IDS und Honeypots

<http://www.net-tex.de/net/ids.html>

krypt

SCHUMACHER, Stefan

Einführung in kryptographische Methoden

<http://www.cryptomancer.de/21c3/>

backup

SCHUMACHER, Stefan
Backupstrategien und Techniken für NetBSD
<http://www.net-tex.de/backup.pdf>

zwick1

ZWICKY, Elizabeth D.
Torture-testing Backup and Archive Programs: Things You Ought to Know But Probably Would Rather Not
Proceedings of the Fifth LISA Conference, USENIX, 1991.
<http://grox.net/doc/zwick1/testsuite/torture.pdf>

zwick2

ZWICKY, Elizabeth D.
Further Torture: More Testing of Backup and Archive Programs
Proceedings of the 17th Large Installation Systems Administration Conference
http://www.usenix.org/events/lisa03/tech/full_papers/zwick2/zwick2.pdf

vse

RUSKE, Sandra & SCHUMACHER, Stefan
*Belegarbeit zur Vorlesung "Verteilte Systementwicklung" —
Zeit in verteilten Systemen*
<http://www-e.uni-magdeburg.de/steschum/vse.html>

RFC1305

MILLS, David L.
Network Time Protocol (Version 3) Specification, Implementation and Analysis
<ftp://ftp.rfc-editor.org/in-notes/rfc1305.pdf>
<ftp://ftp.rfc-editor.org/in-notes/rfc1305.txt>

skey-de

DFN-CERT Services GmbH
Informationsbulletin DIB-94:04 vom 21. Juli 1994: "S/Key"
<http://www.dfn-cert.de/infoserv/dib/dib-9404.html>

rfc1760

HALLER, Neil M.
The S/KEY One-Time Password System
<ftp://ftp.rfc-editor.org/in-notes/rfc1760.txt>
<ftp://ftp.rfc-editor.org/in-notes/rfc1760.pdf>

RFC2289

HALLER, N. & METZ, C. & NESSER, P. & STRAW, M.
A One-Time Password System
<ftp://ftp.rfc-editor.org/in-notes/rfc2289.pdf>
<ftp://ftp.rfc-editor.org/in-notes/rfc2289.txt>

- cgds_{swap} SEDLACIK, Lubomir
encrypted swap with cgd(4) (eMail discussion)
<http://mail-index.netbsd.org/tech-security/2003/04/17/0000.html>
- IEEE₁₇ ALLEN, Julia; CHRISTIE, Alan; MCHUGH, John
 CERT Coordination Center
Defending Yourself: The Role of Intrusion Detection Systems
 „IEEE Software“ #17 Sept/Oct 2000
- eino₄ EINWECHTER, Nathan
Multi-Layer Intrusion Detection Systems
<http://www.securityfocus.com/infocus/1788>
- oh₁₋₁ OUDOT, Laurent; HOLZ, Thorsten
Defeating Honeypots: Network Issues, Part 1 & 2
<http://www.securityfocus.com/infocus/1803>
<http://www.securityfocus.com/infocus/1805>
- scho₄ SCHUMACHER, Stefan
Einführung in kryptographische Methoden
<http://cryptomancer.ath.cx/~stefan/21c3/21c3-kryptographie-paper.pdf>
- scho₃ SCHUMACHER, Stefan
Spass im Netzwerk mit tcpdump & Co.
<http://www-e.uni-magdeburg.de/steschum/tcpdump.pdf>
- gruno₅ GRUNWALD, Lukas
Aufbau und Betrieb von Honeypot-Systemen
http://www.linuxdays.lu/agenda/lxd2005_security_tutorial/Attachment00089954/lxd2005_honeypot.pdf
- niel_{so4} PROVOS, Niels
A Virtual Honeypot Framework
 USENIX Security 2004
<http://www.citi.umich.edu/u/provos/papers/honeyd.pdf>
- pic_{o3} PICKETT, Mark
A Guide to the Honeypot Concept
 GSEC Practical v1.4b
http://www.giac.org/practical/GSEC/Mark_Pickett_GSEC.pdf
- PoDa_{o3} POUGET, F.; DACIER, M.
Honeypot-based Forensics
http://www.eurecom.fr/pouget/papiers/AusCERT_fullpaper_BIS.pdf

Institut Eurecom

- reh03 REHMAN, Rafeeq Ur
Intrusion Detection Systems with Snort
<http://phptr.com/content/images/0131407333/downloads/0131407333.pdf>
Prentice Hall PTR, 2003
- HJSo3 HESS, A.; JUNG, M.; SCHÄFER, G.
Combining Multiple Intrusion Detection and Response Technologies in an Active Networking Based Architecture
<http://www.tkn.tu-berlin.de/publications/papers/dfno3.pdf>
TU Berlin
- Caro3 CARLO, Corbin Del
Intrusion detection evasion: How Attackers get past burglar alarms
<http://cnscenter.future.co.kr/resource/security/ids/1284.pdf>
SANS Institute 2003
- sno23 ROESCH, Martin; GREEN, Chris; Sourcefire Inc.
SnortUsers Manual 2.3.0
http://www.snort.org/docs/snort_manual/
- snof The Snort Core Team
The Snort FAQ
<http://www.snort.org/docs/faq.pdf>
- Scto2 SCOTT, Steven
Threat Management: The State of Intrusion Detection
<http://www.snort.org/docs/threatmanagement.pdf>
- pn98 PTACEK, Thomas; NEWSHAM, Timothy
Insertion, Evasion, and Denial of Service: Eluding Network Intrusion Detection
<http://www.snort.org/docs/idspaper/>
- sr1 SPENNEBERG, Ralf
Sensoren im Netzwerk
Linux-Magazin Security Edition
[http://www.spenneberg.com/Snort\(SecurityEdition\)/2134.html](http://www.spenneberg.com/Snort(SecurityEdition)/2134.html)

INDEX

/dev/null, 38
/dev/rst0, 58
/etc/daily.conf, 40
/etc/daily, 39
/etc/defaults/daily.conf, 40
/etc/dumpdates, 58
/etc/master.passwd, 31
/etc/passwd.conf, 36
/etc/passwd, 36
/etc/rc.conf, 48
GENERIC_VERIEXEC, 61
MAKEDEV, 89
Man-in-the-middle, 18
XScreensaver, 105
Xlockmore-light, 105
Xlockmore, 105
&, 38
anacron, 48
arpwatch, 84
audit-packages, 30
cgd, 89
check_disks, 40
check_mailq, 40
check_network, 40
check_uucp, 40
crontab, 47
cron, 47
dd, 100
expire_news, 40
find_core, 40
fstat(8), 45
full_netstat, 40
ipcs(1), 45
lock, 38
modstat(8), 45
mtree, 63
netstat(1), 43
nfsstat(1), 45
pkg_chk, 30
ps(1), 43
pstat(8), 45
purge_accounting, 40
run_calendar, 40
run_fsck, 40
run_msgs, 40
run_rdist, 40
run_security, 40
run_skeyaudit, 40
screen, 39
send_empty_security, 40
show_remote_fs, 40
stat(1), 45
stderr, 38
stdout, 38
su(1), 30
sudo, 30
sysinst, 106
syslogd, 54
sysstat(1), 43
top(1), 43
verixec, 61
vmstat(1), 45
xautolock, 105

address
 ethernet
 monitoring, 84
AIDE, 65
AIDE
 options, 66
anacron, 48
arpwatch, 84

backdoor, 18
background, 38
backup, 55
backup
 complete, 56
 delta, 56
 differential, 56
 incremental, 56

- media, 57
 - plan, 56
 - strategy, 56
- Buffer Overflow, 20
- cfs, 96
- cfs
 - media, 97
- CGD, 89
- challenge, 37
- challenge/response, 37
- checksums, 58
- chkrootkit, 69
- Cleartext Passwords, 19
- cron, 47, 62
- cron
 - schedule, 47
- Curie-temperature, 100
- dDoS, 19
- Degausser, 100
- Denial of Service, 19
- device nodes, 89
- dictionary attack, 31
- DoS, 19
- encryption
 - symmetric, 99
- erase
 - secure, 100
- file
 - access, 50
 - group, 50
 - mode, 50
 - owner, 50
 - permission, 50
 - sgid, 50
 - suid, 50
- file
 - dereferencing, 100
- file flags, 51
- filesystem
 - cryptographic, 96
- fingerprint, 62
- fingerprints
 - nonmatching, 62
- gateway, 83
- GID, 36
- Group ID, 36
- hacker, 16
- IDS, 23
- integrity
 - file, 65
- interface
 - decrypted, 97
 - file system, 96
- Intrusion Detection System, 23
- kernel, 62
- kernel
 - lean, 29
 - loadable modules, 30
- keylogger, 21
- keystroke
 - record, 21
- lastlog, 70
- LKM, 30, 70
- LKM
 - diable, 30
- loadable kernel modules, 30
- locking a terminal, 38
- login, 36
- login
 - class, 36
 - directory, 36
 - shell, 36
- maintenance, 39
 - daily, 39
- mcrypt, 99
- mtree
 - example, 64
 - keywords, 63
- one time password, 37
- OpenSSL, 34

- passwd, 34
- otp, 37
- package database, 54
- packages
 - check version, 30
 - exploited, 30
- password, 31, 36
 - cracking, 32
 - database, 31
 - encrypted, 31
 - guessing, 31
 - one time, 37
 - strong, 31
- password
 - cipher, 37
 - local, 37
 - NIS, 37
 - YP, 37
- phishing, 20
- pkgsrsc, 54
- ports, 83
- portsentry, 83
- Posix, 50
- process, 38
- process to background, 38
- processes
 - running, 43
- promiscuous mode, 70
- pseudo device, 89
- radiation
 - compromising, 21
 - jammer, 21
 - shielding, 21
- removable media
 - cfs, 97
- response, 37
- rootkit, 18, 61
 - scan for, 69
- route, 83
- S/Key, 37
- SALT, 35
- Script-Kiddies, 16
- security
 - alert team, 115
 - officer, 115
- security advisory, 30
- security levels, 51
- security policy, 20
- services
 - required, 29
 - start, 29
- social engineering, 20, 31
- syslog, 54
- system
 - fingerprint, 63
- system
 - complexity, 29
 - sets, 29
- system messages, 54
- tape device, 58
- terminal
 - locking, 38
- terminal multiplexer, 39
- trojan, 61
- UID, 36
- User ID, 36
- userland, 62
- utmp, 70
- verified executables, 61
- Virus, 17
- vulnerability list, 55
- wipe, 101
- Worm, 18
- wtmpx, 70
- X
 - locking, 105
- X session, 106
 - kill, 106